



WARWICK

Advanced Computer Graphics

Monte Carlo Integration

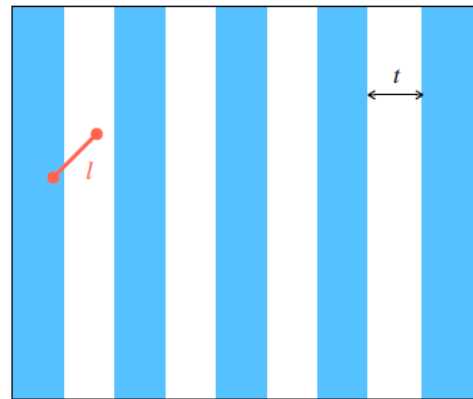
Dr. Tom Bashford-Rogers

Randomized Algorithms

- ▶ Solve a problem using randomness
 - Why?
 - Intractable using deterministic algorithms
 - Can be faster on average
- ▶ Two classes
 - Las Vegas (Correct solution, e.g. quicksort)
 - Monte Carlo (Approximate solution)

Monte Carlo History

- ▶ Buffon's Needle (18th century)
 - “What is the probability that a needle will lie across two strips”
- ▶ Student (1900s)
- ▶ Fermi (1930s)



$$Pr(X = onstrips) = \frac{2l}{\pi t}$$

$$l < t$$

Monte Carlo History

- ▶ Ulam, Von Neumann, Metropolis (1940s)
- ▶ General science (1950s – Present)



Monte Carlo Example

▶ Estimate pi

▶ Given a unit circle with area

$$\pi r^2 = \pi$$

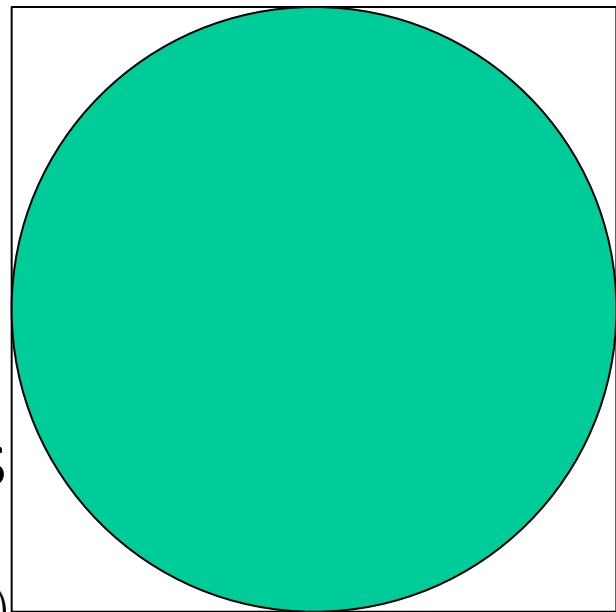
▶ And a square with area 4

▶ Ratio of circle to square areas

$$\frac{\pi}{4}$$

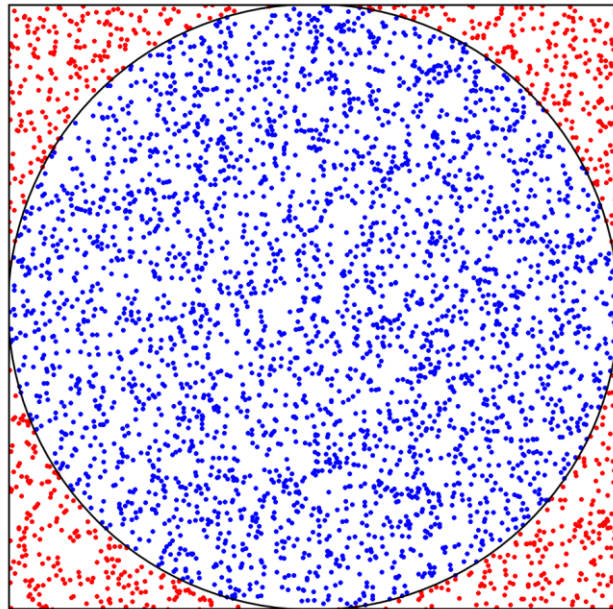
$(-1, -1)$

$(1, 1)$



Monte Carlo Example

- ▶ Estimate pi
- ▶ Estimate with Monte Carlo
 - Generate points in square
 - Check if inside circle
 - Calculate ratio of inside vs outside
 - Estimate pi: $\frac{\text{inside}}{N} = \frac{\pi}{4}$, so $\pi = \frac{4 \times \text{inside}}{N}$



Background

► Random variable

- Function X that assigns a real number to each outcome in the sample space $\mathcal{D}$

$$X : \mathcal{D} \rightarrow \mathbb{R}$$

- Discrete and continuous



Background

- ▶ Discrete Random variable
 - Has a countable set of values
 - Distributed according to a Probability Mass Function (PMF)

$$p_X(x) = \Pr(X = x)$$



Background

- ▶ Continuous Random variable
 - Has an uncountable range of values from a domain
 - Distributed according to a Probability Density Function (PDF)

$$p(x) \quad \text{with} \quad \int_{\mathcal{D}} p(x) dx = 1$$



Background

► Note on domain

– If d is reals

$$\int_D p(x) dx = \int_{-\infty}^{\infty} p(x) dx = 1$$

– But can be anything of interest

• So

$$\Pr(a \leq X \leq b) = \int_a^b p(x) dx$$



Background

► Cumulative Distribution Function

- Gives the probability that X takes a value less than or equal to y

$$CDF(x) = \Pr(X \leq x) = \int_{lb}^x p(t) dt,$$

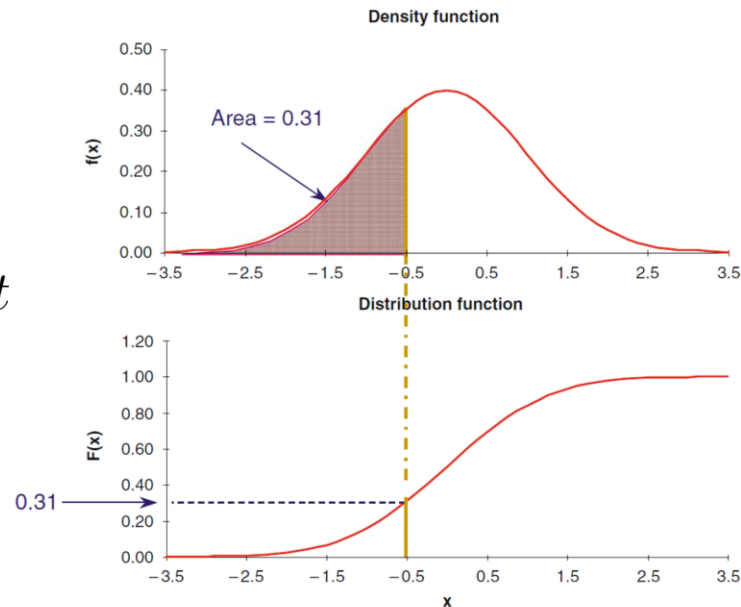
- Where lb is the lower bound
- Monotonically increasing



Background

► Cumulative Distribution Function

$$CDF(x) = \Pr(X \leq x) = \int_{-\infty}^x p(t) dt$$



Background

► Expected Value

- A weighted average of all possible values

$$E[X] = \sum_x x \Pr(X = x)$$

$$E[X] = \int_{\mathcal{D}} x p(x) dx$$



Background

- ▶ Functions of a random variable
 - If for $X : \mathcal{D} \rightarrow \mathbb{R}$ we have a function $f : \mathbb{R} \rightarrow \mathbb{R}$
 - Then $f(X)$ is also a random variable



Background

► Expected Value of a function

- For any measurable function $f : \mathcal{D} \rightarrow \mathbb{R}$ we are interested in the behaviour of $f(x)$ when x is distributed according to $p(x)$

$$E[f(x)] = \int_{\mathcal{D}} f(x) p(x) dx$$

- This is the quantity we are most interested in!

Background

► Variance

- Measures the spread around the mean

$$V[f(x)] = E\left[\left(f(x) - E[f(x)]\right)^2\right]$$

- Equivalently

$$V[f(x)] = E\left[f(x)^2\right] - \left(E[f(x)]\right)^2$$

- Used to quantify the error $SE = \sqrt{V[f(x)]}$

Background

► Example (Uniform on a domain of reals) $f(x) = x$

$$p(x) = \begin{cases} \frac{1}{b-a}, & x \in [a, b], \\ 0, & \text{otherwise} \end{cases}$$

$$E[x] = \int_a^b f(x) \cdot \frac{1}{b-a} dx = \frac{a+b}{2}$$

$$V[x] = \int_a^b \left(f(x) - \frac{a+b}{2} \right)^2 \frac{1}{b-a} dx = \frac{(b-a)^2}{12}$$

Monte Carlo

- ▶ Want to calculate $I = \int_a^b f(x) dx$ but do not have an analytical solution
- ▶ Let $p(x) = \frac{1}{b-a}$ for $x \in [a, b]$
- ▶ Using the definition of the expected value

$$E[f(X)] = \int_a^b f(x) p(x) dx = \int_a^b f(x) \cdot \frac{1}{b-a} dx = \frac{1}{b-a} \int_a^b f(x) dx$$

Monte Carlo

- ▶ Rearrange $I = \int_a^b f(x) dx = (b - a) E[f(X)]$
- ▶ Now we need to compute $E[f(X)]$
- ▶ Use (Strong) Law of Large Numbers
 - Given N i.i.d. random samples $x_1, x_2, \dots, x_N$

$$I_N = \frac{1}{N} \sum_{i=1}^N f(x_i) \rightarrow E[f(X)] \quad \text{as } N \rightarrow \infty$$

Monte Carlo

► Combine $I_N = \frac{b-a}{N} \sum_{i=1}^N f(x_i)$

► Converges: Unbiased Estimator

$$E[I_N] = E \left[\frac{b-a}{N} \sum_{i=1}^N f(x_i) \right]$$

$$= \frac{b-a}{N} \sum_{i=1}^N E[f(x_i)]$$

$$= \frac{b-a}{N} \sum_{i=1}^N \int_a^b f(x) p(x) dx$$

$$= \frac{1}{N} \sum_{i=1}^N \int_a^b f(x) dx$$

$$= \int_a^b f(x) dx$$

Monte Carlo

- ▶ Can generalize $p(x)$ to any distribution

$$I = \frac{1}{N} \sum_{i=1}^N \frac{f(x_i)}{p(x_i)}$$

- ▶ Very useful later
- ▶ Note we can generalize to a domain $\mathcal{D}$: $x_i \in \mathcal{D}$



Monte Carlo

► Proof as before

$$\begin{aligned} E[F] &= E \left[\frac{1}{N} \sum_{i=1}^N \frac{f(x_i)}{p(x_i)} \right] \\ &= \frac{1}{N} \sum_{i=1}^N \int_{\mathcal{D}} \frac{f(x)}{p(x)} p(x) dx \\ &= \int_{\mathcal{D}} f(x) dx \end{aligned}$$

Monte Carlo

► PDF has to obey certain properties

$$p(x) \geq 0 \quad \forall x \in \mathcal{D}$$

$$\int_{\mathcal{D}} p(x) dx = 1$$

$$p(x) > 0 \quad \text{whenever } f(x) \neq 0,$$



Monte Carlo

► Variance of Monte Carlo Estimate

- Let $Y_i = f(x_i)$, $i = 1, \dots, N$
- Assume finite variance of each sample:

$$\sigma^2 = V[Y_i] = E[f(x)^2] - (E[f(x)])^2$$

- Want variance of estimate

$$V[Y_N] = V\left[\frac{1}{N} \sum_{i=1}^N Y_i\right] = \frac{1}{N^2} V\left[\sum_{i=1}^N Y_i\right] = \frac{1}{N^2} \sum_{i=1}^N V[Y_i] = \frac{1}{N^2} N \sigma^2 = \frac{\sigma^2}{N}$$

Monte Carlo

► Variance of Monte Carlo Estimate with N

- Variance: decreases linearly $\mathcal{O}(\frac{1}{N})$

- Error

$$SE = \sqrt{V[f(x)]} = \sqrt{\frac{\sigma^2}{N}} = \frac{\sigma}{\sqrt{N}}$$

- Decreases with rate $\mathcal{O}(\frac{1}{\sqrt{N}})$

- 4X samples for ½ error!

Monte Carlo

- ▶ Need to be able to draw samples from the PDF: $x_1, x_2, \dots, x_N \sim p$
- ▶ Need to map a uniform random variable to a random variable from desired distribution
- ▶ Main methods:
 - Inverse Transform Sampling
 - Rejection Sampling

Monte Carlo

- ▶ Need to be able to draw samples from the PDF

- ▶ Start with uniform random numbers

$$\xi \sim \text{Uniform}(0, 1)$$

- ▶ We will discuss these later



Monte Carlo

► Inverse Transform Sampling (aka CDF Inversion)

- Recall $CDF(x) = \Pr(X \leq x) = \int_{lb}^x p(t) dt,$
- CDF: the accumulated probability up to each value
- Invert probability to yield sample

Monte Carlo

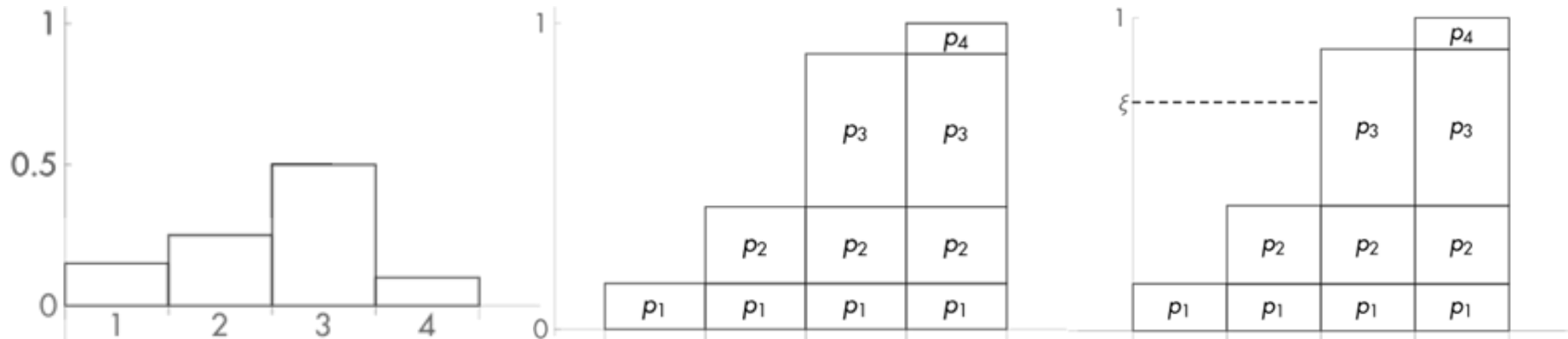
► Inverse Transform Sampling (aka CDF Inversion)

- Then $x = CDF^{-1}(\xi)$.
- Requires the CDF can be inverted
 - Not always possible
 - Often involves involved derivations!



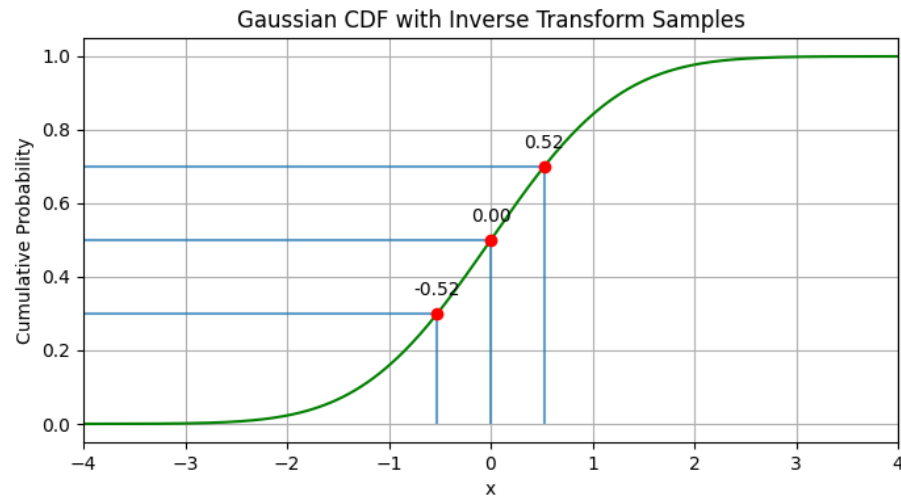
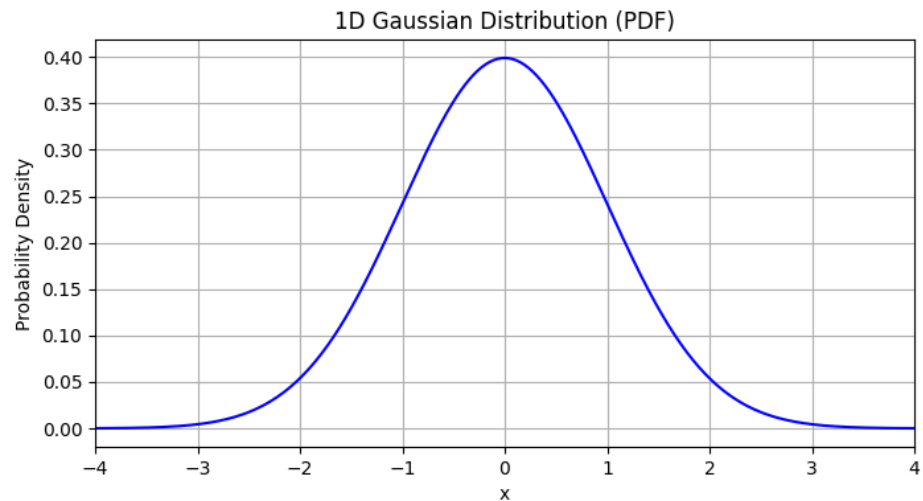
Monte Carlo

► Inverse Transform Sampling (Discrete Case)



Monte Carlo

► Inverse Transform Sampling (Continuous)



Monte Carlo

► Inverse Transform Sampling

– Example (power distribution $c x^n$ in domain $[0, 1]$)

- Normalize $\int_0^1 c x^n dx = c \cdot \frac{1}{n+1} = 1 \implies c = n+1$
- Calculate PDF $p(x) = (n+1)x^n, \quad x \in [0, 1]$
- Calculate CDF $CDF(x) = \int_0^x (n+1)t^n dt = x^{n+1}$
- CDF Inversion $\xi = x^{n+1} \implies x = \xi^{\frac{1}{n+1}}$
- Sampling Draw $\xi \sim \mathcal{U}(0, 1)$ and set $x = \xi^{\frac{1}{n+1}}$

Monte Carlo

► Inverse Transform Sampling

– Example (power distribution $c x^2$ in domain $[2, 3]$)

- Normalize $\int_2^3 c x^2 dx = c \left[\frac{x^3}{3} \right]_2^3 = c \left(\frac{27 - 8}{3} \right) \implies c = \frac{3}{19}$
- Calculate PDF $p(x) = \frac{3}{19} x^2$
- Calculate CDF $CDF(x) = \int_2^x \frac{3}{19} t^2 dt = \frac{x^3 - 2^3}{19} = \frac{x^3 - 8}{19}$
- CDF Inversion $\xi = \frac{x^3 - 8}{19} \implies x = \sqrt[3]{19\xi + 8}$
- Sampling Draw $\xi \sim \mathcal{U}(0, 1)$ and set $x = \sqrt[3]{19\xi + 8}$

Monte Carlo

► Inverse Transform Sampling

– Example (exponential distribution $c e^{-ax}$ in $[0, \infty]$)

- Normalize $\int_0^\infty c e^{-ax} dx = c \left[\frac{1}{a} \right] = \frac{c}{a} = 1 \implies c = a$
- Calculate PDF $p(x) = a e^{-ax}$
- Calculate CDF $CDF(x) = \int_0^x a e^{-at} dt = 1 - e^{-ax}$
- CDF Inversion $\xi = 1 - e^{-ax} \implies x = -\frac{1}{a} \ln(1 - \xi) = -\frac{1}{a} \ln(\xi)$
- Sampling Draw $\xi \sim \mathcal{U}(0, 1)$ and set $x = -\frac{1}{a} \ln(\xi)$

Monte Carlo

- ▶ Transforming between distributions
 - If there is a 1-1 map between distributions
 - Can transform between PDFs

$$p_y(y) \frac{dy}{dx} = p_x(x) \implies p_y(y) = \left| \frac{dy}{dx} \right|^{-1} p_x(x)$$

Monte Carlo

- ▶ Transforming between distributions
 - Given an invertible and differentiable transformation T
 - Can transform random variable X : $Y = T(X)$
 - i.e. $y = T(x) \Leftrightarrow x = T^{-1}(y)$
 - Density change via:
$$p_Y(y) = \left| \frac{dy}{dx} \right|^{-1} p_X(x)$$

Monte Carlo

► Transforming between distributions

– We have already used this implicitly

- e.g. transforming uniform to exponential distribution

$$p_x(x) = a e^{-ax} \quad x \in [0, \infty] \quad y = CDF_x(x) = 1 - e^{-ax}$$

$$p_y(y) = 1 \quad y \in [0, 1]$$

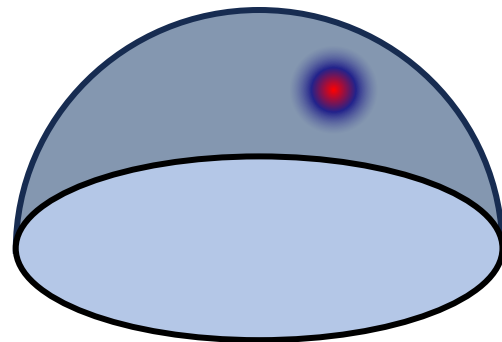
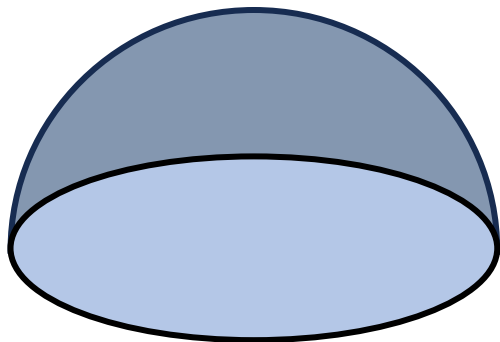
$$\left| \frac{dy}{dx} \right| = a e^{-ax} \implies \left| \frac{dy}{dx} \right|^{-1} = \frac{1}{a e^{-ax}}$$

$$p_y(y) = \left| \frac{dy}{dx} \right|^{-1} p_x(x)$$

$$p_y(y) = \frac{1}{a e^{-ax}} \cdot a e^{-ax} = 1$$

Monte Carlo

- ▶ What about more complicated distributions?



Monte Carlo

► Multidimensional Distributions

- Described as a density of two or more variables $p(x, y)$
- Sometimes separable $p(x, y) = p(x)p(y)$
 - Can sample each distribution independently
 - Mostly not separable in practice

Monte Carlo

► Multidimensional Distributions

- How to sample
- Calculate Marginal Distribution $p(x) = \int p(x, y) dy$
 - Integrate out one variable
- Conditional Distribution $p(y|x) = \frac{p(x, y)}{p(x)}$
 - Can compute this from step 1

Monte Carlo

► Transforming between multidimensional distributions

- Generalize to nd case $p_Y(y) = p_Y(T(x)) = \frac{p_X(x)}{|J_T(x)|}$
- Jacobian (matrix of partial derivatives)

$$J_T(x) = \begin{bmatrix} \frac{\partial T_1(x)}{\partial x_1} & \cdots & \frac{\partial T_1(x)}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial T_n(x)}{\partial x_1} & \cdots & \frac{\partial T_n(x)}{\partial x_n} \end{bmatrix}$$

Monte Carlo

► Transforming between distributions

– Example: Polar Coordinates $x = r \cos(\theta)$

$$y = r \sin(\theta)$$

– Form Jacobian $J_T = \begin{bmatrix} \frac{\partial x}{\partial r} & \frac{\partial x}{\partial \theta} \\ \frac{\partial y}{\partial r} & \frac{\partial y}{\partial \theta} \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -r \sin(\theta) \\ \sin(\theta) & r \cos(\theta) \end{bmatrix}$

– Calculate absolute value of determinant $|J_T| = r$

– Map between densities $p(r, \theta) = r p(x, y)$

Monte Carlo

► Transforming between distributions

- Example: Spherical Coordinates
$$\begin{aligned}x &= r \sin(\theta) \cos(\phi) \\y &= r \sin(\theta) \sin(\phi) \\z &= r \cos(\theta)\end{aligned}$$
- Calculate Jacobian and absolute value of determinant $|J_T| = r^2 \sin(\theta)$
- Map between densities $p(r, \theta, \phi) = r^2 \sin(\theta) p(x, y, z)$
 - For unit sphere $p(\theta, \phi) = \sin(\theta) p(x, y, z)$

Monte Carlo

► Multidimensional Distributions

– Example: Uniformly sampling a hemisphere

- Solid angle PDF: $\int_{\Omega} p(\omega) d\omega = 1, \Rightarrow c \int_{\Omega} d\omega, \Rightarrow p(\omega) = \frac{1}{2\pi}$

- Convert to spherical coordinates: $p(\theta, \phi) = \frac{\sin(\theta)}{2\pi}$
 - $\sin(\theta)$ from previous slide

- Marginal PDF: $p(\theta) = \int_0^{2\pi} \frac{\sin(\theta)}{2\pi} d\phi = \sin(\theta)$

- Conditional PDF: $p(\phi|\theta) = \frac{1}{2\pi}$

Monte Carlo

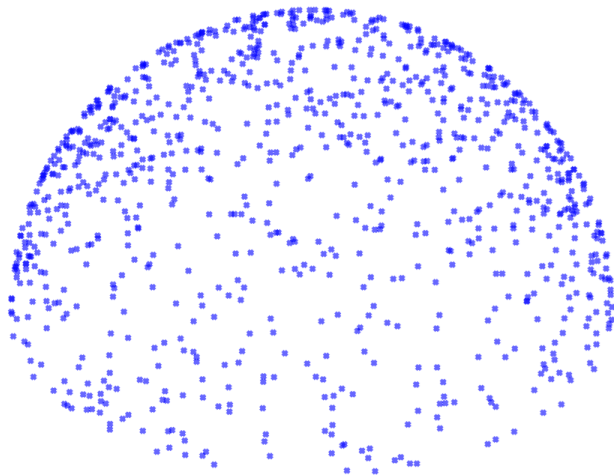
► Multidimensional Distributions

– Example: Uniformly sampling a hemisphere

- Calculate CDFs: $CDF(\theta) = \int_0^\theta \sin(\theta') d\theta' = 1 - \cos(\theta)$
 $CDF(\phi) = \int_0^\phi \frac{1}{2\pi} d\phi' = \frac{\phi}{2\pi}$
- Invert CDFs for sampling: $\theta = \cos^{-1}(1 - \xi_1) = \cos^{-1}(\xi_1)$
 $\phi = 2\pi\xi_2$

Monte Carlo

- ▶ Multidimensional Distributions
 - Example: Uniformly sampling a hemisphere



Monte Carlo `SphericalCoordinates::sphericalToWorld(theta, phi);`

► Multidimensional Distributions

- Implementation:

- Note these are in a z-up coordinate system

- In Sampling.h, find class

`SamplingDistributions`

- Fill in methods:

```
static Vec3 uniformSampleHemisphere(float r1, float r2)
```

```
static float uniformHemispherePDF(const Vec3 wi)
```



Monte Carlo

► Multidimensional Distributions

– Example: Uniformly sampling a sphere

- Solid angle PDF: $\int_{\mathcal{S}} p(\omega) d\omega = 1, \Rightarrow c \int_{\mathcal{S}} d\omega, \Rightarrow p(\omega) = \frac{1}{4\pi}$
- Convert to spherical coordinates: $p(\theta, \phi) = \frac{\sin(\theta)}{4\pi}$
 - $\sin(\theta)$ from previous slide
- Marginal PDF: $p(\theta) = \int_0^{2\pi} \frac{\sin(\theta)}{4\pi} d\phi = \frac{\sin(\theta)}{2}$
- Conditional PDF: $p(\phi|\theta) = \frac{1}{2\pi}$

Monte Carlo

► Multidimensional Distributions

– Example: Uniformly sampling a sphere

- Calculate CDFs: $CDF(\theta) = \int_0^\theta \frac{\sin(\theta')}{2} d\theta' = \frac{1 - \cos(\theta)}{2}$

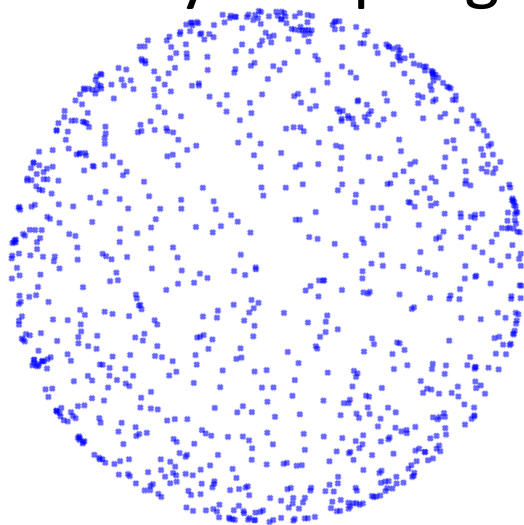
$$CDF(\phi) = \int_0^\phi \frac{1}{2\pi} d\phi' = \frac{\phi}{2\pi}$$

- Invert CDFs for sampling: $\theta = \cos^{-1}(1 - 2\xi_1)$

$$\phi = 2\pi\xi_2$$

Monte Carlo

- ▶ Multidimensional Distributions
 - Example: Uniformly sampling a sphere



Monte Carlo

► Multidimensional Distributions

- Implementation
- In Sampling.h, find class

SamplingDistributions

- Fill in methods:

```
static Vec3 uniformSampleSphere(float r1, float r2)
```

```
static float uniformSpherePDF(const Vec3& wi)
```



Monte Carlo

► Multidimensional Distributions

- Example: Cosine sampling a hemisphere $p(\omega) \propto \cos(\theta)$
 - Need $\int_{\Omega} p(\omega) d\omega = 1$
 - Calculate normalizing constant $p(\omega) = c \cos(\theta)$
 - Convert to spherical coordinates

$$p(\theta, \phi) = \int_0^{2\pi} \int_0^{\frac{\pi}{2}} c \cos(\theta) \sin(\theta) d\theta d\phi$$

Monte Carlo

► Multidimensional Distributions

– Example: Cosine sampling a hemisphere

• So
$$p(\theta, \phi) = c 2\pi \int_0^{\frac{\pi}{2}} \cos(\theta) \sin(\theta) d\theta = 1$$

$$u = \sin(\theta), du = \cos(\theta) d\theta$$

$$p(\theta, \phi) = c 2\pi \int_0^1 u du = 1 \Rightarrow c = \frac{1}{\pi}$$

$$p(\theta, \phi) = \frac{1}{\pi} \cos(\theta) \sin(\theta)$$

Monte Carlo

► Multidimensional Distributions

– Example: Cosine sampling a hemisphere $p(\omega) = \frac{\cos(\theta)}{\pi}$

- Marginal PDF:

$$p(\theta) = \int_0^{2\pi} \frac{\cos(\theta) \sin(\theta)}{\pi} d\phi = 2 \cos(\theta) \sin(\theta)$$

- Conditional PDF:

$$p(\phi|\theta) = \frac{1}{2\pi}$$

Monte Carlo

► Multidimensional Distributions

– Example: Cosine sampling a hemisphere

- Calculate CDFs: $CDF(\theta) = \int_0^\theta 2 \cos(\theta') \sin(\theta') d\theta' = \sin^2(\theta)$

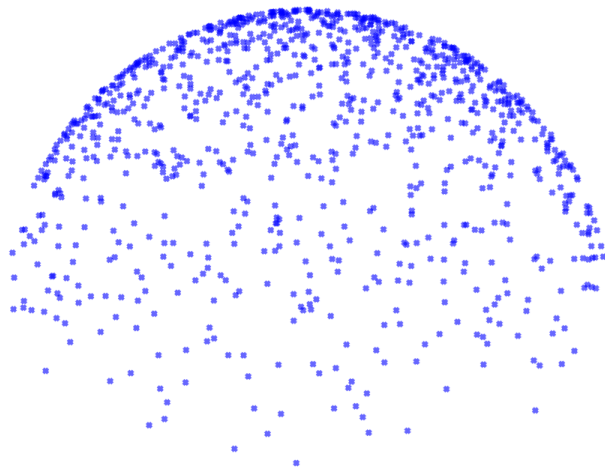
$$CDF(\phi) = \int_0^\phi \frac{1}{2\pi} d\phi' = \frac{\phi}{2\pi}$$

- Invert CDFs for sampling: $\theta = \cos^{-1}(\sqrt{\xi_1})$

$$\phi = 2\pi\xi_2$$

Monte Carlo

- ▶ Multidimensional Distributions
 - Example: Cosine sampling a hemisphere



Monte Carlo

► Multidimensional Distributions

- Implementation
- In Sampling.h, find class

SamplingDistributions

- Fill in methods:

```
static Vec3 cosineSampleHemisphere(float r1, float r2)
```

```
static float cosineHemispherePDF(const Vec3 wi)
```

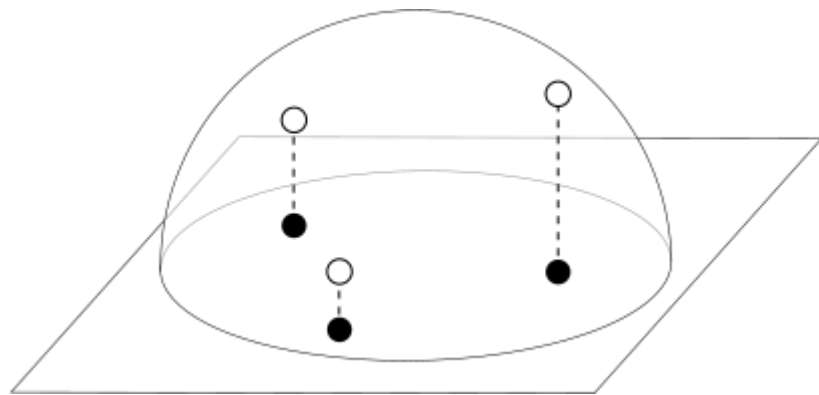


Monte Carlo

► Multidimensional Distributions

– Example: Cosine sampling a hemisphere

- Alternative Method:
- Uniform sample disk
- Project to hemisphere
- Malley's Method



Monte Carlo

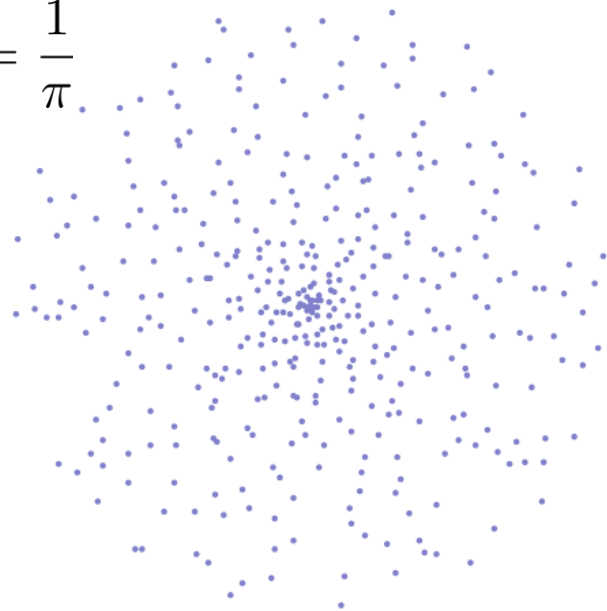
- ▶ Multidimensional Distributions
 - Uniform sample unit disk $p(x, y) = \frac{1}{\pi}$
 - Convert to polar coordinates $p(\phi, r) = \frac{r}{\pi}$
 - Factor of r comes from Jacobian



Monte Carlo

► Multidimensional Distributions

- Uniform sample unit disk $p(x, y) = \frac{1}{\pi}$
- Naïve (incorrect) approach
 - Sample $r = \xi_1$ and $\phi = 2\pi\xi_2$
 - Samples clustered in the centre



Monte Carlo

► Multidimensional Distributions

- Uniform sample unit disk $p(x, y) = \frac{1}{\pi}$, $p(\phi, r) = \frac{r}{\pi}$
- Marginal distribution $p(r) = \int_0^{2\pi} p(\phi, r) d\phi = 2r$
- Conditional distribution $p(\theta_d|r) = \frac{p(\phi, r)}{p(r)} = \frac{1}{2\pi}$
- Calculate CDFs

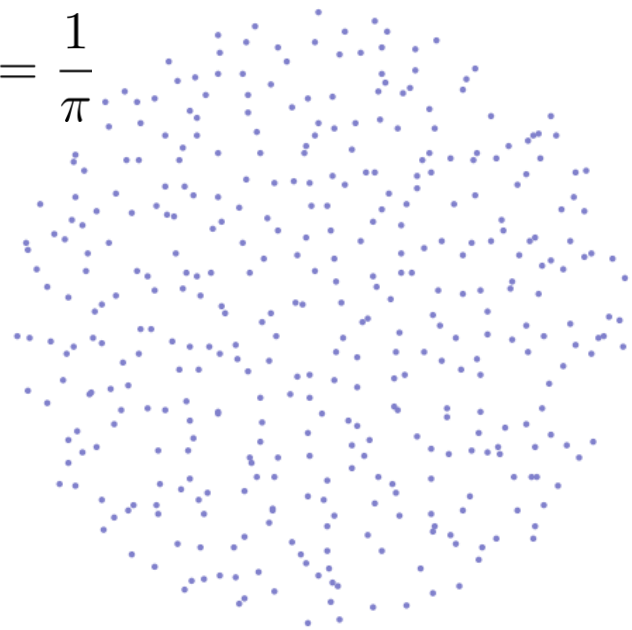
$$CDF(r) = \int_0^r 2t dt = r^2$$

$$CDF(\phi) = \int_0^\phi \frac{1}{2\pi} dt = \frac{\phi}{2\pi}$$

Monte Carlo

► Multidimensional Distributions

- Uniform sample unit disk $p(x, y) = \frac{1}{\pi}$
- Sampling $r = \sqrt{\xi_1}, \quad \phi = 2\pi\xi_2$



Monte Carlo

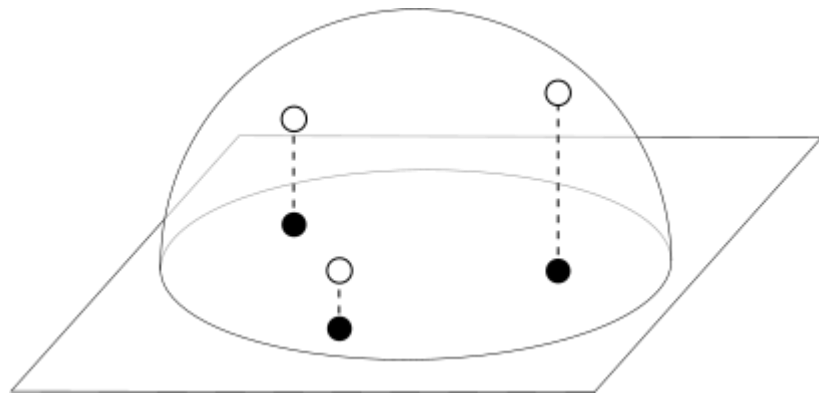
- ▶ Multidimensional Distributions
 - Example: Cosine sampling a hemisphere
 - Malley's Method

$$r = \sqrt{\xi_1}, \quad \phi = 2\pi\xi_2$$

$$x = r \cos(\phi)$$

$$y = r \sin(\phi)$$

$$z = \sqrt{1 - (x^2 + y^2)}$$



Monte Carlo

► Multidimensional Distributions

– Malley's Method: Why does this work?

- Need map from disk to hemisphere

$$(r, \phi) = (\sin(\theta), \phi) \mapsto (\theta, \phi)$$

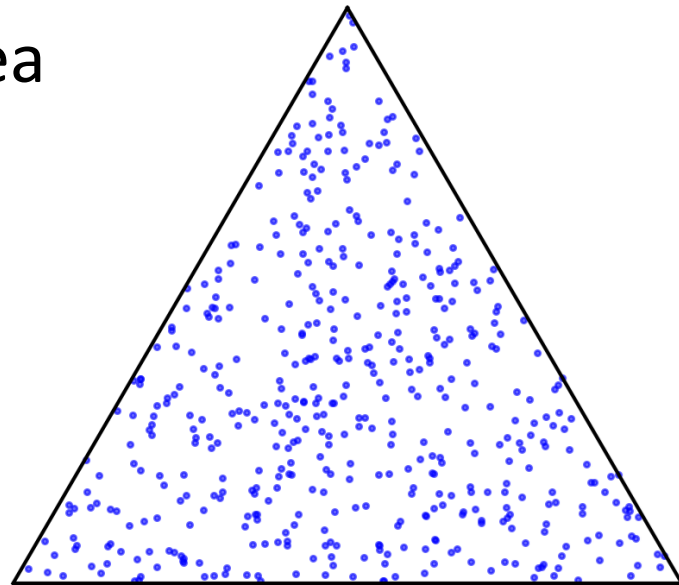
- Jacobian Determinant $\begin{vmatrix} \cos(\theta) & 0 \\ 0 & 1 \end{vmatrix} = \cos(\theta)$

- So $p(\theta, \phi) = \cos(\theta)p(r, \phi) = \frac{\cos(\theta)r}{\pi} = \frac{\cos(\theta)\sin(\theta)}{\pi}$

Monte Carlo

- ▶ Sampling Triangles
 - PDF of picking point w.r.t. area

$$p(x_l) = \frac{1}{A}$$



Monte Carlo

- ▶ Uniformly sampling triangle area
 - Sample barycentric coordinates to generate point $P = \alpha P_0 + \beta P_1 + \gamma P_2$
 - Generate two random numbers ξ_1 and ξ_2
 - Generate barycentric coordinates via transform:

$$\alpha = 1 - \sqrt{\xi_1}, \quad \beta = \xi_2 \sqrt{\xi_1}, \quad \gamma = 1 - (\alpha + \beta)$$

Monte Carlo

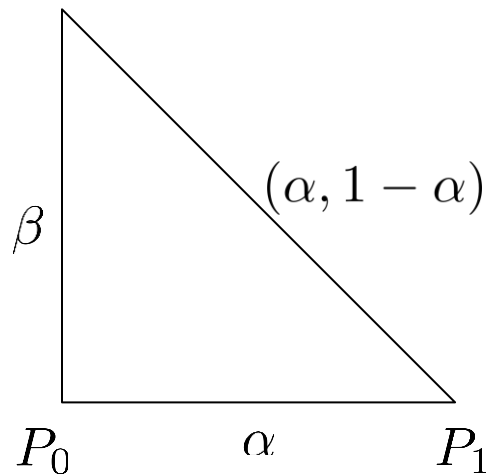
► Uniformly sampling triangle area

- Given a 2d triangle with vertices P_2

$$P_0 = (0, 0), P_1 = (1, 0), P_2 = (0, 1)$$

- Constant pdf:

$$p(\alpha, \beta) = 2$$



Monte Carlo

► Uniformly sampling triangle area

- Marginal Density

$$p(\alpha) = \int_0^{1-\alpha} p(\alpha, \beta) d\beta = 2(1 - \alpha)$$

- Conditional Density

$$p(\beta|\alpha) = \frac{p(\alpha, \beta)}{p(\alpha)} = \frac{2}{2(1 - \alpha)} = \frac{1}{1 - \alpha}$$



Monte Carlo

► Uniformly sampling triangle area

- Calculate CDFs $CDF(\alpha) = \int_0^\alpha p(\alpha') d\alpha' = 2\alpha - \alpha^2$
 $CDF(\beta) = \int_0^\beta p(\beta'|\alpha) d\beta' = \frac{\beta}{1-\alpha}$
- Use for sampling via CDF inversion and simplifying

$$\alpha = 1 - \sqrt{\xi_1}$$

$$\beta = \xi_2 \sqrt{\xi_1}$$

Monte Carlo

► Multidimensional Distributions

– Example: Tabulated Distributions (Discrete)

- Given by $F[u_i, v_j]$ $i \in [0, w]$
 $j \in [0, h]$
- Assumed $F[u_i, v_j] = \text{constant}$ across bin
- Map $(u, v) \in [0, 1]^2$ to $(u_{i'}, v_{j'}) \in [0, N_w] \times [0, N_h]$
- i.e. $u_{i'} = \lfloor u \cdot N_w \rfloor$ and $v_{j'} = \lfloor v \cdot N_h \rfloor$

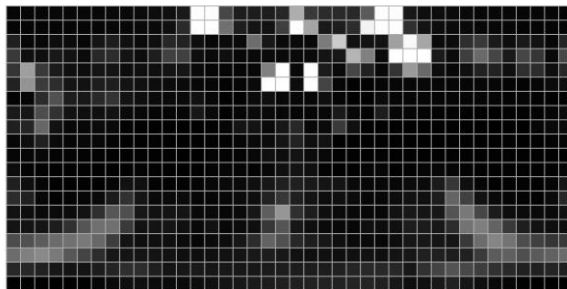
Monte Carlo

► Multidimensional Distributions

– Example: Tabulated Distributions

- PDF:

$$p(u, v) = \frac{F[u_{i'}, v_{j'}]}{\frac{1}{N_w N_h} \sum_{i=0}^{N_w-1} \sum_{j=0}^{N_h-1} F[u_i, v_j]}$$



Monte Carlo

► Multidimensional Distributions

– Example: Tabulated Distributions

- Marginal PDF:
 - Sum of rows

$$p(v) = \frac{\frac{1}{N_w} \sum_{i=0}^{N_w-1} F[u_i, v_{j'}]}{\frac{1}{N_w N_h} \sum_{i=0}^{N_w-1} \sum_{j=0}^{N_h-1} F[u_i, v_j]}$$

- Conditional PDF:

$$p(u|v) = \frac{\frac{F[u_{i'}, v_{j'}]}{\frac{1}{N_w N_h} \sum_{i=0}^{N_w-1} \sum_{j=0}^{N_h-1} F[u_i, v_j]}}{p(v)}$$

Monte Carlo

► Multidimensional Distributions

– Example: Tabulated Distributions

- Compute CDFs:

- One CDF for v (column): $CDF(v) = \sum_{k=0}^{v_j} p(v_k)$

- One CDF per u (row): $CDF(u|v) = \sum_{l=0}^{u_i} p(u_l|v_k)$

Monte Carlo

► Multidimensional Distributions

– Example: Tabulated Distributions

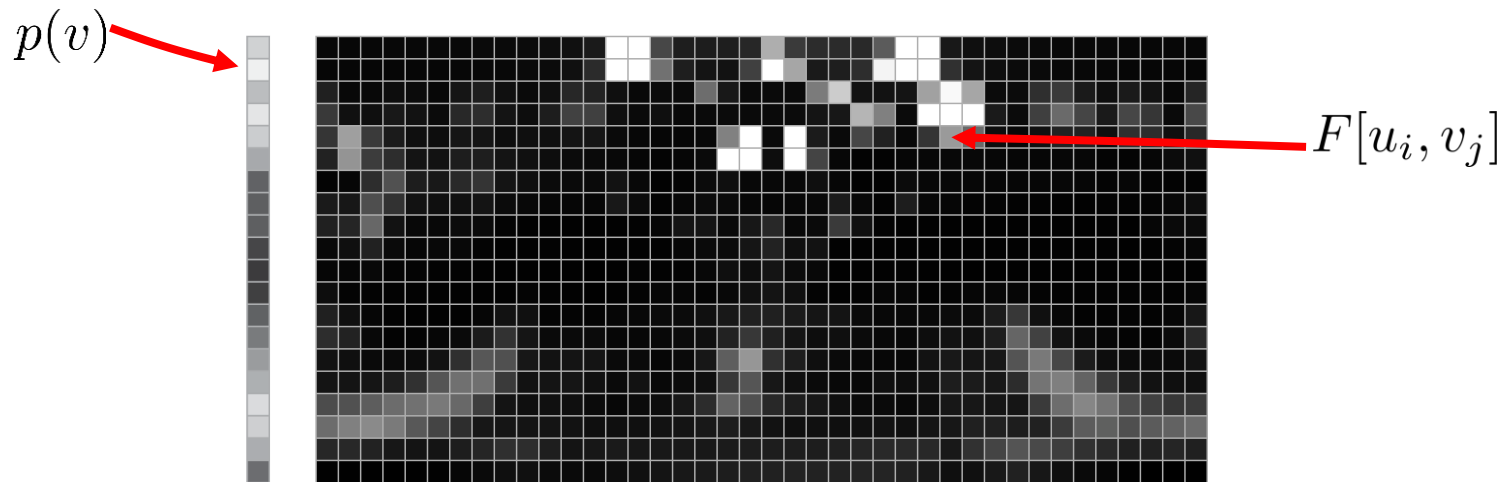
- Sampling

- Sample v via searching: $CDF(v)$
- Usually use binary search ($\mathcal{O}(\log n)$ vs $\mathcal{O}(n)$)
- But needs more precomputation, usually worth it
- Then sample u from $CDF(u|v)$



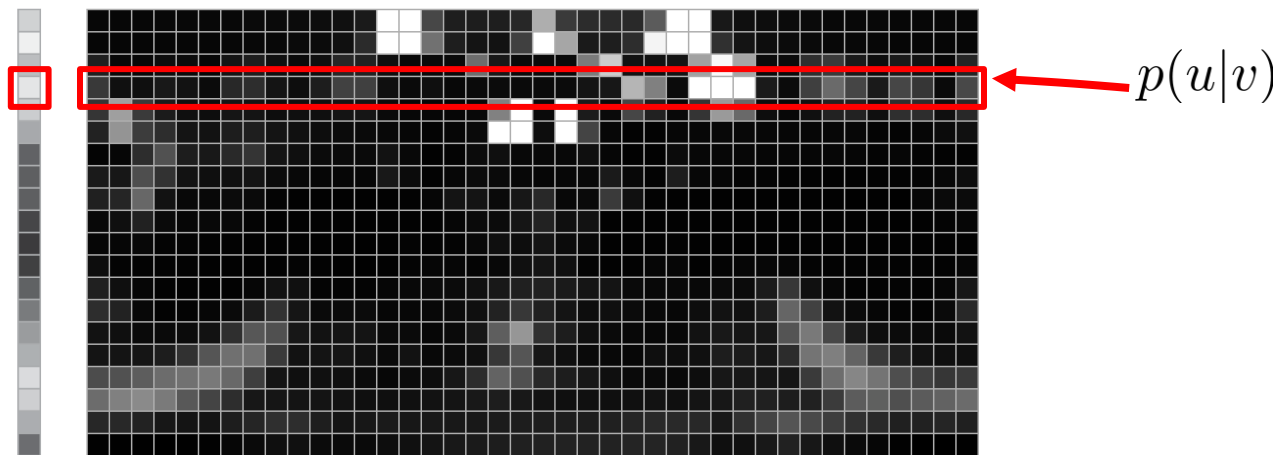
Monte Carlo

- ▶ Multidimensional Distributions
 - Example: Tabulated Distributions



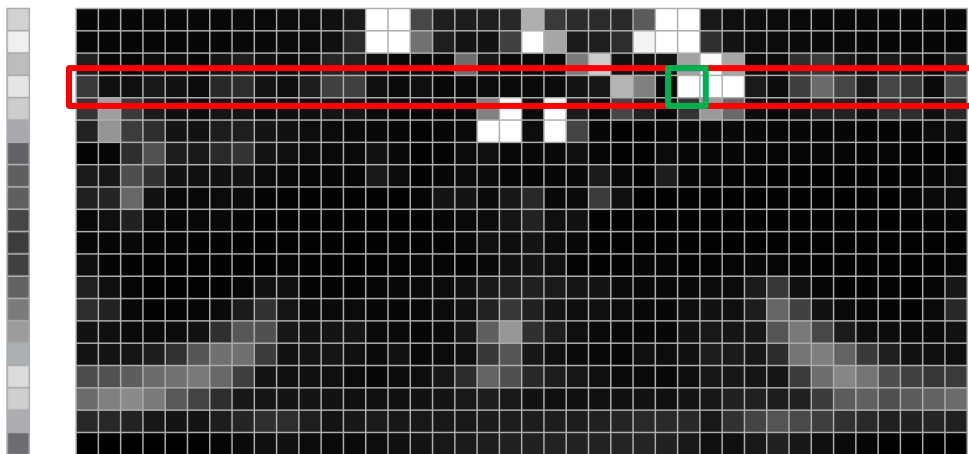
Monte Carlo

- ▶ Multidimensional Distributions
 - Example: Tabulated Distributions



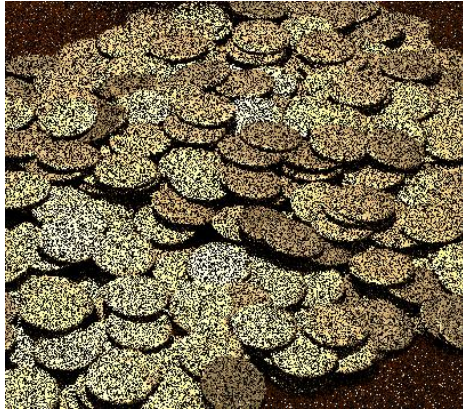
Monte Carlo

- ▶ Multidimensional Distributions
 - Example: Tabulated Distributions



Variance

► Noise from random sampling



1 SPP



4 SPP



16 SPP



64 SPP



Variance Reduction

- ▶ Want to reduce variance
 - Need fewer samples for same quality images
 - Main issue with
 - Speed up computation
 - Higher FPS for games
 - Important for film industry (power consumption)



Variance Reduction

- ▶ Better sample placement in the domain
 - Importance Sampling
- ▶ Better spread of samples in the domain
 - Stratified Sampling
- ▶ Better weighting of samples
 - Multiple Importance Sampling and Control Variates
- ▶ Better generation of samples $\xi \sim \mathcal{U}(0, 1)$

Importance Sampling

- ▶ Recall the standard Monte Carlo estimator

$$I = \frac{1}{N} \sum_{i=1}^N \frac{f(x_i)}{p(x_i)}$$

- ▶ If $p(x_i)$ is similar to $f(x_i)$ then variance is reduced
 - Known as Importance Sampling

Importance Sampling

► Core idea

- Draw more samples where the integrand has a high value and fewer where value is lower
- Need to weight samples to account for non-uniform sampling



Importance Sampling

► Core idea

– Ideally $p(x_i) \propto f(x_i)$ i.e.: $p(x_i) = \frac{f(x_i)}{\int_{\mathcal{D}} f(x) dx} = \frac{f(x_i)}{c}$

$$I = \frac{1}{N} \sum_{i=1}^N \frac{f(x_i)}{\frac{f(x_i)}{c}} = \frac{c}{N} \sum_{i=1}^N \frac{f(x_i)}{f(x_i)} = c$$

– If we could do this we would not need Monte Carlo!

Importance Sampling

- ▶ However, if we can find $p(x_i)$ which is similar to $f(x_i)$ but analytically integrable then can use this distribution
- ▶ Many, many options
 - Often power law curves, gaussians, exp / log, analytically integrable basis functions



Importance Sampling

► Example

– If $f(x) = x^3$ and $\mathcal{D} = [0, 1]$, known value 0.25

– Compare:

- Uniform sampling $p(x) = 1$

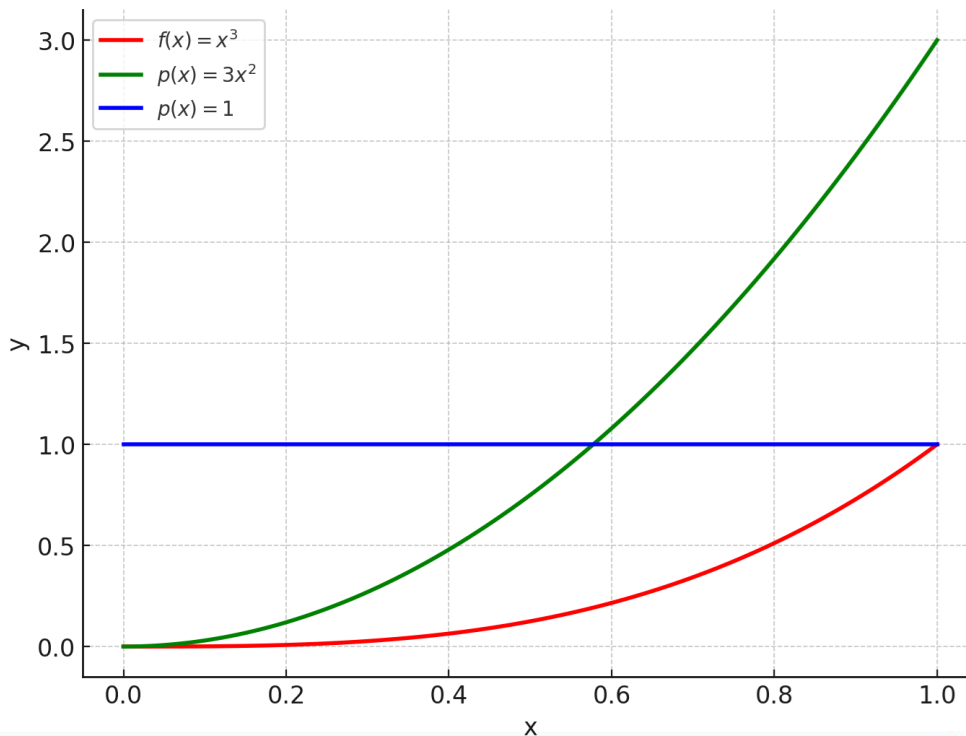
$$\xi \sim \mathcal{U}(0, 1)$$

- Importance sampling similar distribution: $p(x) = 3x^2$

$$x = \xi^{\frac{1}{3}}$$

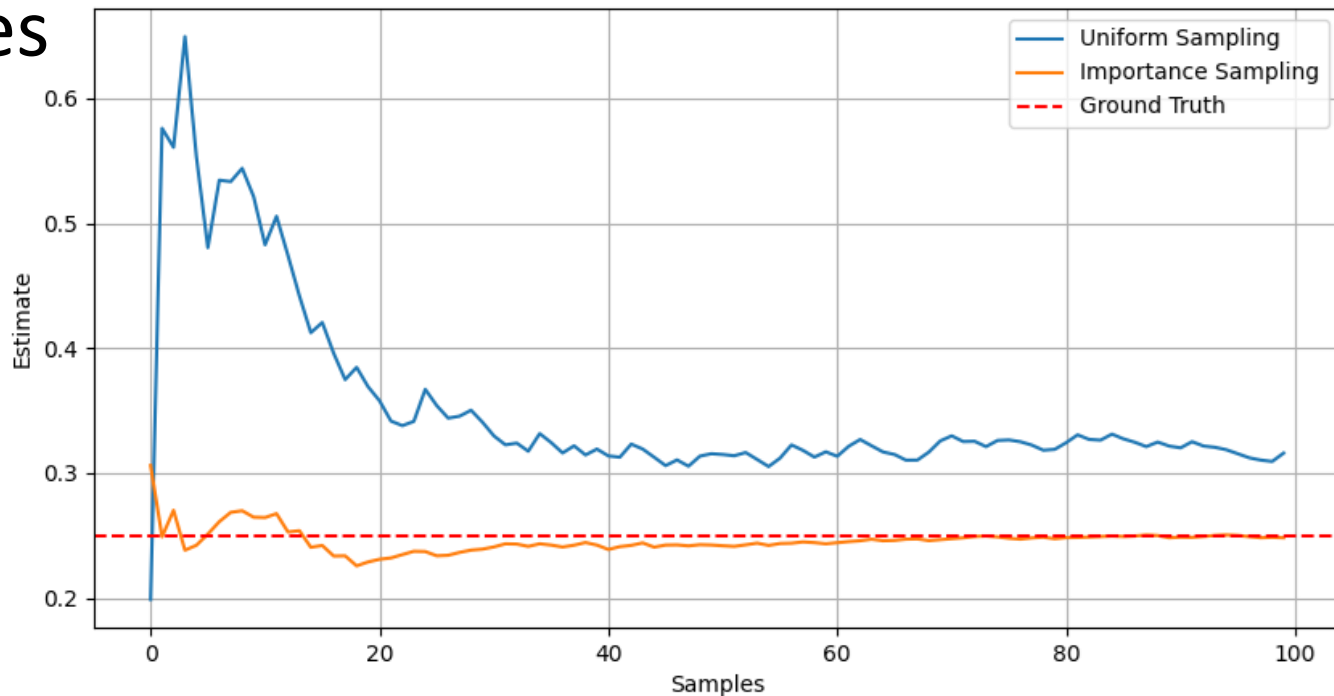
Importance Sampling

► Example



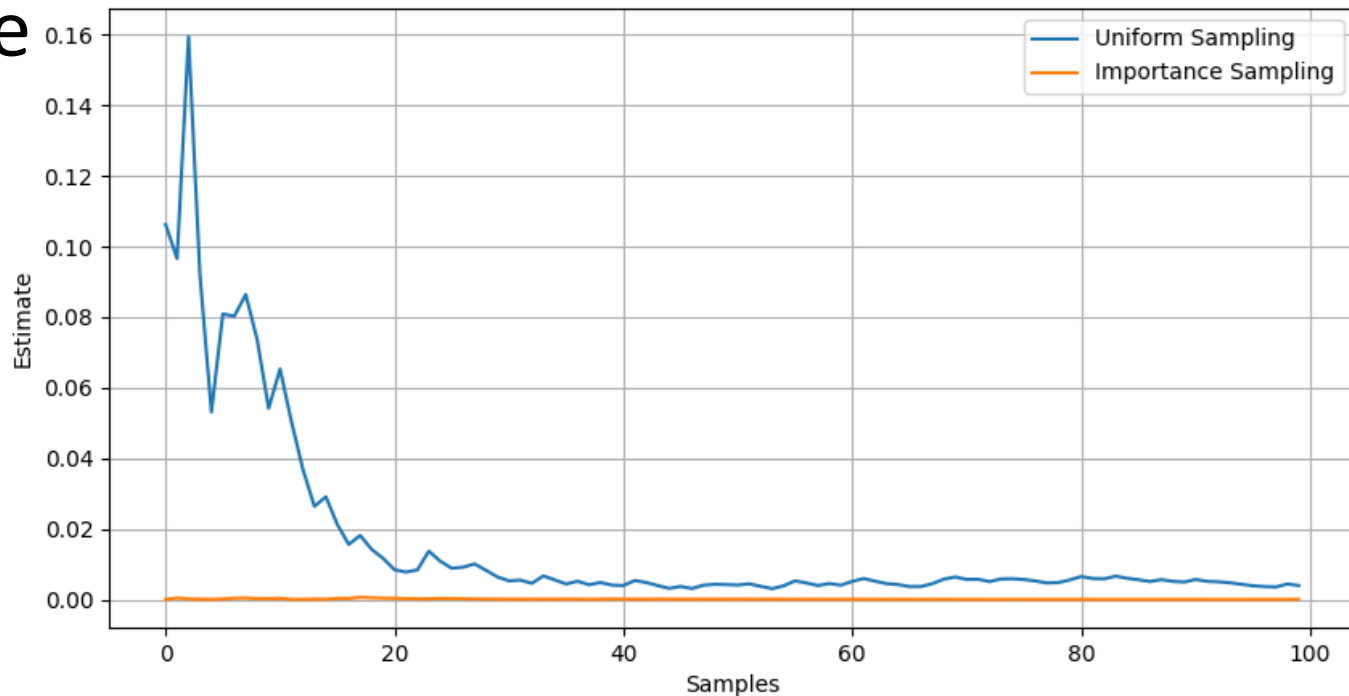
Importance Sampling

► Estimates



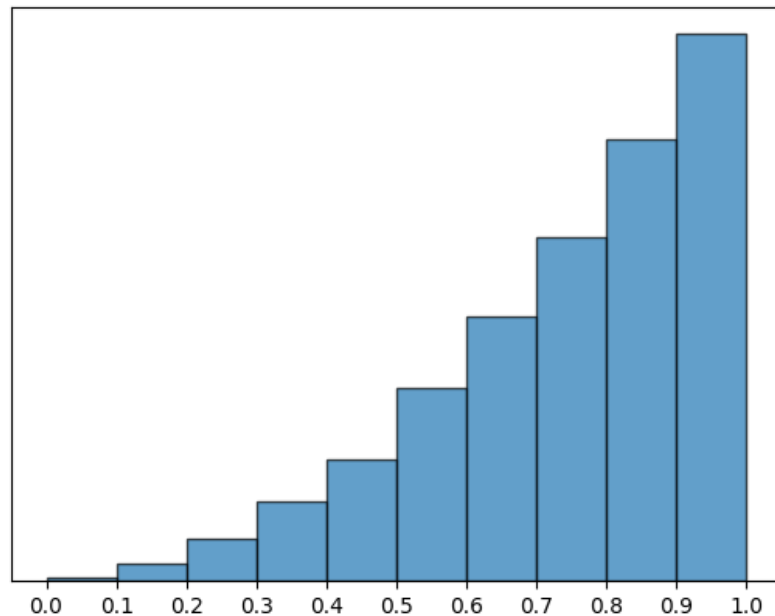
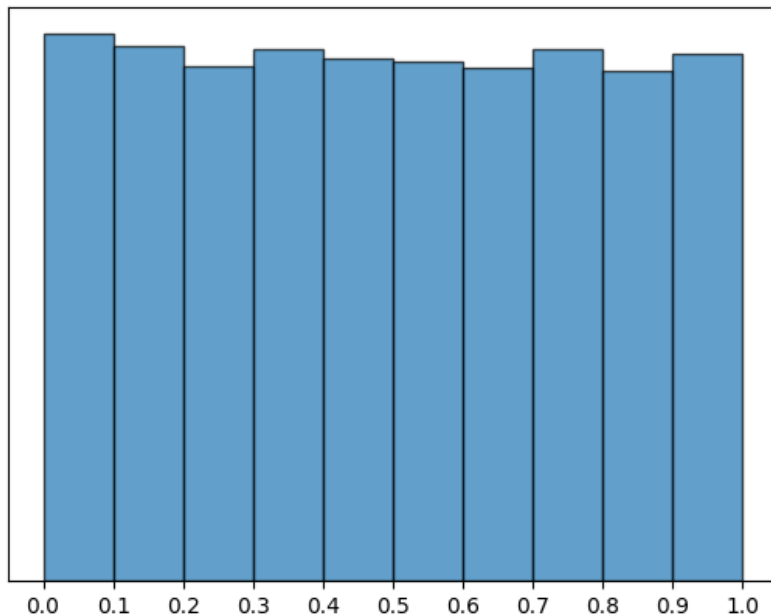
Importance Sampling

► Variance



Importance Sampling

▶ Sample Placement



Importance Sampling

► If $f(x_i)$ is a product, we can sample a term of the product

– e.g. if $f(x_i) = g(x_i)h(x_i)$ and we can sample proportional to $h(x_i)$:

$$I = \frac{1}{N} \sum_{i=1}^N \frac{g(x_i)h(x_i)}{c h(x_i)} = \frac{1}{N} \sum_{i=1}^N \frac{g(x_i)}{c}$$

– i.e. variance from $g(x_i)$ rather than $g(x_i)h(x_i)$



Importance Sampling

► Limitations

- If $p(x_i)$ is a bad match for $f(x_i)$ then variance can increase
- Have to be careful about choices!

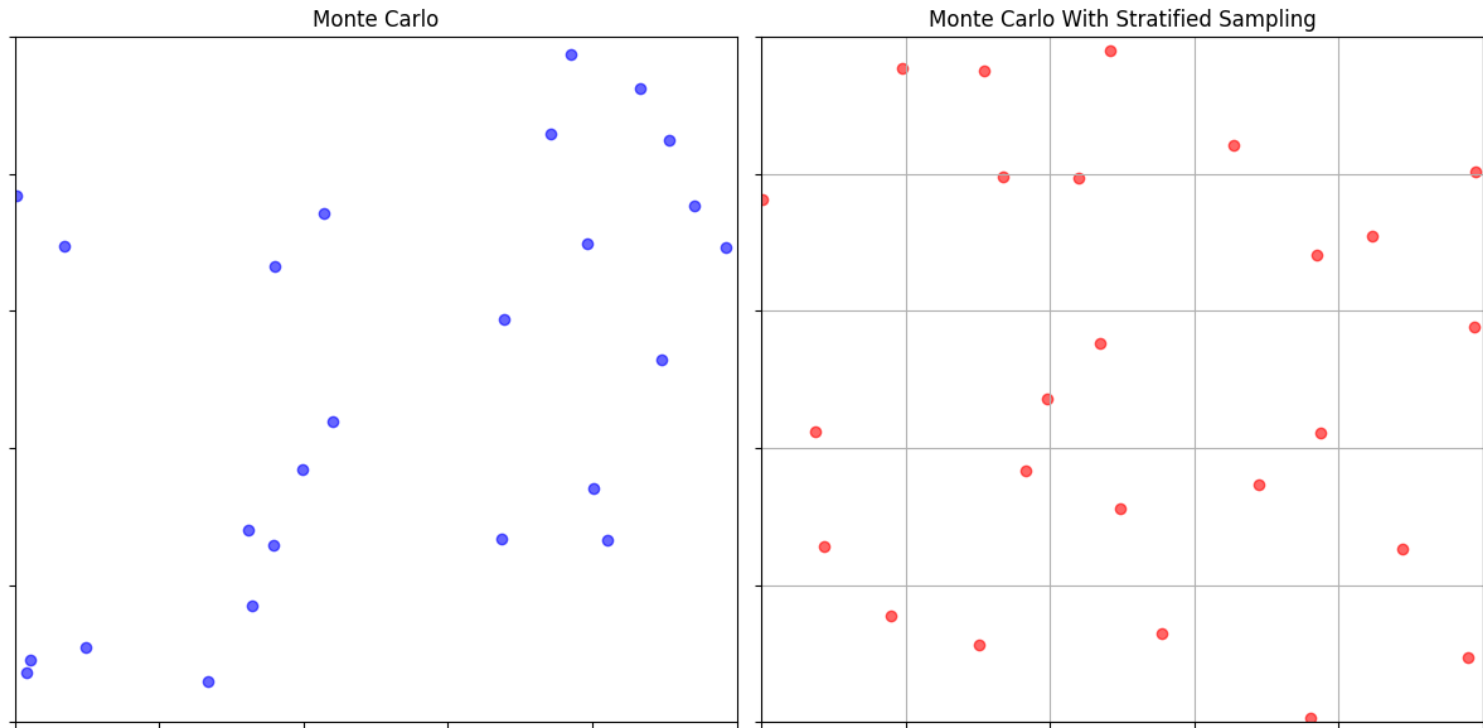


Stratified Sampling

- ▶ Split domain $\mathcal{D}$ into non overlapping regions $\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_M$:
$$\bigcup_{i=1}^M \mathcal{D}_i = \mathcal{D}$$
- ▶ Place sample in each region
- ▶ Guarantees good coverage of the domain



Stratified Sampling



Stratified Sampling

► Estimate in strata: $\int_{\mathcal{D}_i} f(x) dx$

$$I_i = \frac{1}{N_i} \sum_{j=1}^{N_i} \frac{f(x_{i,j})}{p_i(x_{i,j})}$$

► Overall estimate: $I = \sum_{i=1}^M v_i I_i$

– v_i is proportion of samples in stratum i

Stratified Sampling

► Variance:

$$V[I] = \sum_{i=1}^M v_i^2 \frac{\sigma_i^2}{N_i}$$

► Compare with standard Monte Carlo variance ($\frac{\sigma^2}{N}$):

$$\sum_{i=1}^M v_i^2 \frac{\sigma_i^2}{N_i} \leq \frac{\sigma^2}{N}$$

Control Variates

- ▶ If we have a known integrable function $g(x)$ similar to $f(x)$
- ▶ Can add the estimate the difference to the known integral: $\left[\int_{\mathcal{D}} f(x) - \alpha g(x) dx \right] + \alpha \int_{\mathcal{D}} g(x) dx$
 - Note, if $\alpha = 0$ then standard Monte Carlo
- ▶ Lower variance if $g(x)$ is well correlated



Control Variates

► Estimator: $I_{CV} = \frac{1}{N} \sum_{i=1}^N \frac{f(x_i) - \alpha g(x_i)}{p(x_i)} + \alpha I_g$

– Where $I_g = \int_{\mathcal{D}} g(x) dx$

► Value for alpha

– Optimal: $\alpha_{opt} = \frac{Cov[f, g]}{V[g]}$

- Not possible in practice but can estimate (or guess!)

Control Variates

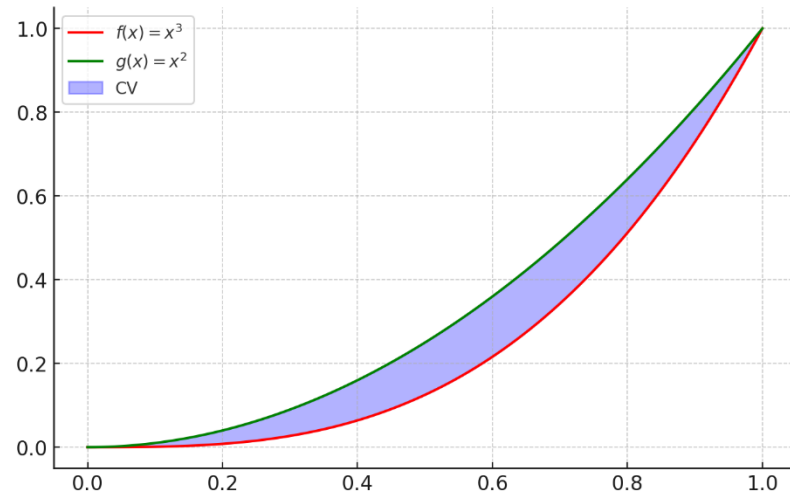
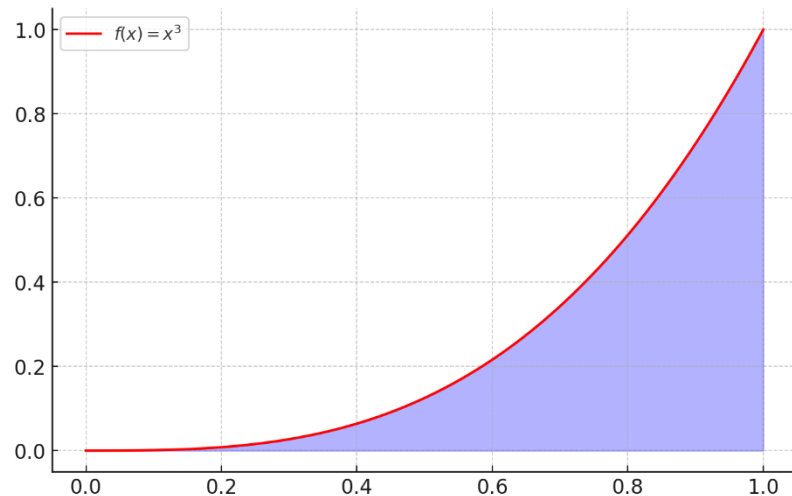
► Variance: $V[I_{CV}] = V[F](1 - \rho^2)$

- Correlation coefficient $\rho = \frac{\text{Cov}[f, g]}{\sqrt{V[f]V[g]}}$
- Variance reduction if positively correlated ($\rho > 0$)
 - But extra cost of evaluating $g(x)$

Control Variates

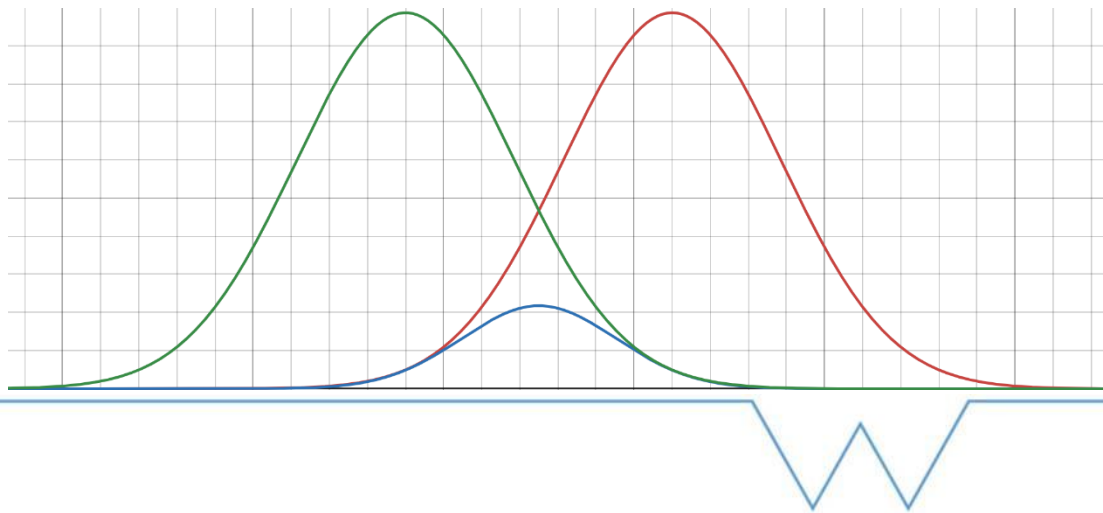
► Example

– If $f(x) = x^3$, $g(x) = x^2$ and $\alpha = 1$



Multiple Importance Sampling

- ▶ Consider the case of $\int_{\mathcal{D}} f(x)g(x)dx$
- ▶ And we have good distributions in the same domain from which to sample $f(x)$ and $g(x)$:
 - $p_f(x)$ and $p_g(x)$



Multiple Importance Sampling

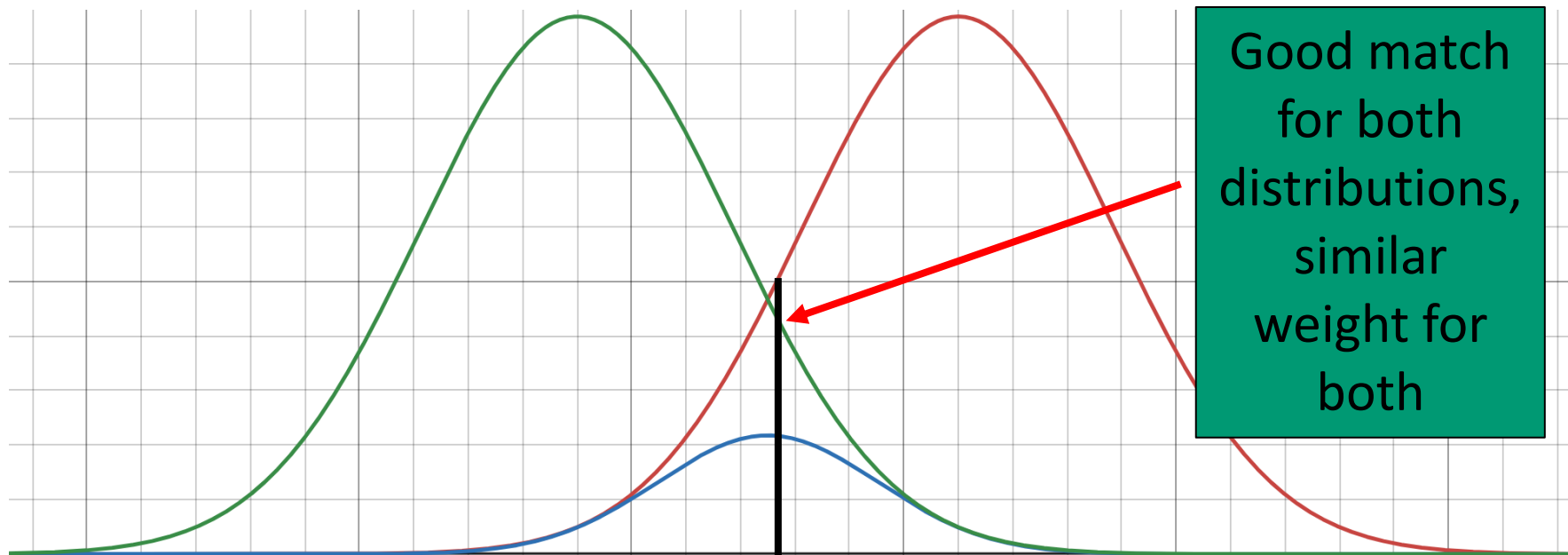
- ▶ If we only use $p_f(x)$: $\frac{1}{N} \sum_{i=1}^N \frac{f(x_i)g(x_i)}{p_f(x_i)} = \frac{1}{N} \sum_{i=1}^N c_f g(x_i)$
- ▶ Or if we only use $p_g(x)$: $\frac{1}{N} \sum_{i=1}^N \frac{f(x_i)g(x_i)}{p_g(x_i)} = \frac{1}{N} \sum_{i=1}^N c_g f(x_i)$
- ▶ Variance proportional to $g(x)$ and $f(x)$ respectively
- ▶ We can do better

Multiple Importance Sampling

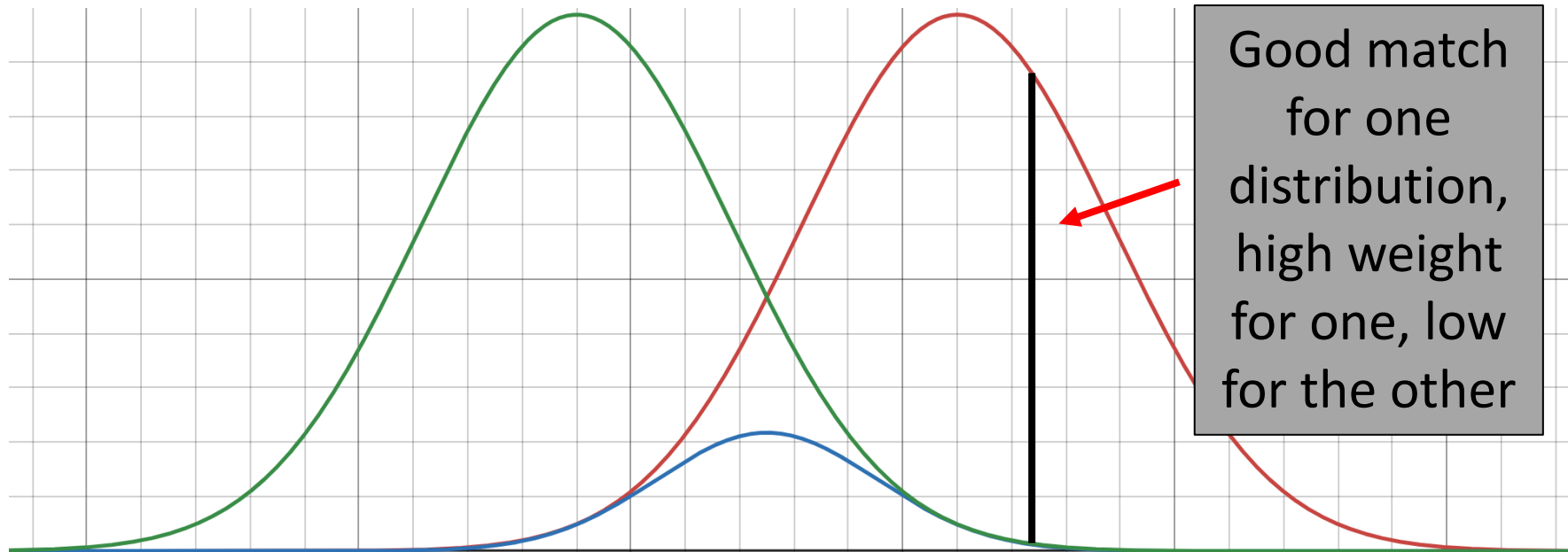
- ▶ Pick a distribution
 - Deterministically or stochastically
- ▶ Sample distribution
- ▶ Weight resulting sample
 - Take into account that the other distribution could have been used



Multiple Importance Sampling



Multiple Importance Sampling



Multiple Importance Sampling

- ▶ Weight for j 'th distribution w_j
 - Any weight can be used if $\sum_{j=1}^M w_j = 1$
- ▶ Heuristics used to calculate weights
 - Balance Heuristic $w_j = \frac{p_j}{\sum_{k=1}^M p_k}$
 - Power Heuristic $w_j = \frac{p_j^\beta}{\sum_{k=1}^M p_k^\beta}$, usually $\beta = 2$
 - Max heuristic $w_j = \max\{p_1, p_2, \dots, p_M\}$

Multiple Importance Sampling

- ▶ Pick a distribution
- ▶ Deterministically
 - Can use fixed number of samples and incorporate into weight
- ▶ Stochastically
 - Randomly choose distribution and include probability of picking distribution

Multiple Importance Sampling

► Deterministic: fixed samples for each N_f and N_g

► Weight (balance heuristic): $w_j = \frac{N_j p_j}{\sum_{k=1}^M N_k p_k}$

► Estimator: $\frac{1}{N_f} \sum_{i=1}^{N_f} w_f \frac{f(x_i)g(x_i)}{p_f(x_i)} + \frac{1}{N_g} \sum_{i=1}^{N_g} w_g \frac{f(x_i)g(x_i)}{p_g(x_i)}$

– Where $w_f = \frac{N_f p_f(x_i)}{N_f p_f(x_i) + N_g p_g(x_i)}$, $w_g = \frac{N_g p_g(x_i)}{N_f p_f(x_i) + N_g p_g(x_i)}$

Multiple Importance Sampling

▶ Insert weight into equation

▶ Estimator:

$$\frac{1}{N_f} \sum_{i=1}^{N_f} \frac{N_f p_f(x_i)}{N_f p_f(x_i) + N_g p_g(x_i)} \frac{f(x_i)g(x_i)}{p_f(x_i)} + \frac{1}{N_g} \sum_{i=1}^{N_g} \frac{N_g p_g(x_i)}{N_f p_f(x_i) + N_g p_g(x_i)} \frac{f(x_i)g(x_i)}{p_g(x_i)}$$

▶ Simplify

$$\sum_{i=1}^{N_f} \frac{f(x_i)g(x_i)}{N_f p_f(x_i) + N_g p_g(x_i)} + \sum_{i=1}^{N_g} \frac{f(x_i)g(x_i)}{N_f p_f(x_i) + N_g p_g(x_i)}$$


Multiple Importance Sampling

- ▶ Stochastically select $p_f(x)$ with probability c_f
 - And $p_g(x)$ with probability c_g
- ▶ Weight (balance heuristic):
$$w_j = \frac{c_j p_j}{\sum_{k=1}^M c_k p_k}$$
- ▶ Single sample estimator:
 - If $p_f(x)$ selected: $w_f \frac{f(x_i)g(x_i)}{c_f p_f(x_i)}$
 - If $p_g(x)$ selected: $w_g \frac{f(x_i)g(x_i)}{c_g p_g(x_i)}$

Multiple Importance Sampling

- ▶ Insert weight into equation
- ▶ Single sample estimator:

$$\frac{c_f p_f(x_i)}{c_f p_f(x_i) + c_g p_g(x_i)} \frac{f(x_i)g(x_i)}{c_f p_f(x_i)} \quad \frac{c_g p_g(x_i)}{c_f p_f(x_i) + c_g p_g(x_i)} \frac{f(x_i)g(x_i)}{c_g p_g(x_i)}$$

- ▶ Both simplify to $\frac{f(x_i)g(x_i)}{c_f p_f(x_i) + c_g p_g(x_i)}$



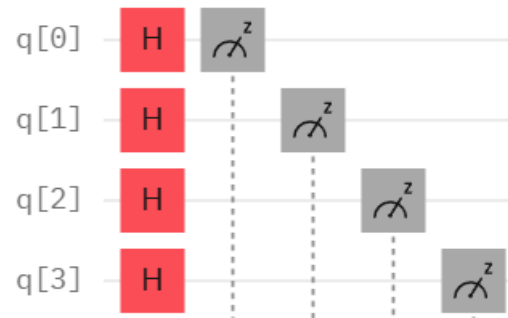
Monte Carlo

- ▶ Generating Uniform Random Numbers
- ▶ Sources of randomness
 - Physical processes
 - Algorithms



Random Numbers

- ▶ Physical processes
 - Atmospheric noise ([link](#), slow)
 - Hardware noise (rdrand x86, uses thermal properties, slow)
 - Quantum Randomness



Random Numbers

► Algorithms:

- Pseudo-Random Number Generation (PRNG)
- Generate a sequence of numbers that approximate true randomness
- Based on a seed number
- There are many algorithms!



Random Numbers

► What makes a PRNG good?

- Uniformity: Numbers should be evenly distributed
- Independence: Should not exhibit predictable patterns
- Determinism: Sequence is reproducible from seed
- Long Period: The sequence should not repeat too quickly
- Efficiency: Should generate numbers quickly



Random Numbers

▶ Linear Congruential Generator

$$X_{n+1} = (aX_n + c) \mod m$$

- a is a multiplier
- c is a constant
- m is the range (i.e. the max value)

▶ Seed via X_0



Random Numbers

► Linear Congruential Generator

- c and m should be relatively prime
- Conditions on a (e.g. $a - 1$ should multiple of 4)

$$X_{n+1} = (aX_n + c) \mod m$$

$$a = 5$$

$$c = 3$$

$$m = 16$$

$$1. X_1 = (5 \times 7 + 3) \mod 16 = 6$$

$$2. X_2 = (5 \times 6 + 3) \mod 16 = 1$$

$$3. X_3 = (5 \times 1 + 3) \mod 16 = 8$$

$$4. X_4 = (5 \times 8 + 3) \mod 16 = 11$$

Random Numbers

► Linear Congruential Generator

– It is:

- Fast
- Simple
- Low memory usage

– Is it good?

- No



Random Numbers

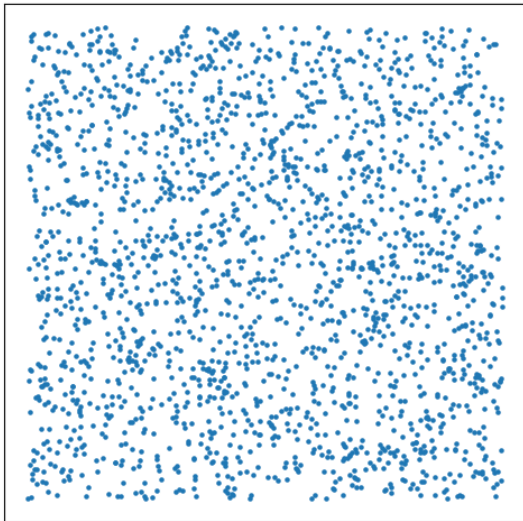
- ▶ Linear Congruential Generator
 - Poor randomness
 - Short period
 - Correlations in high dimensions (bad for rendering)



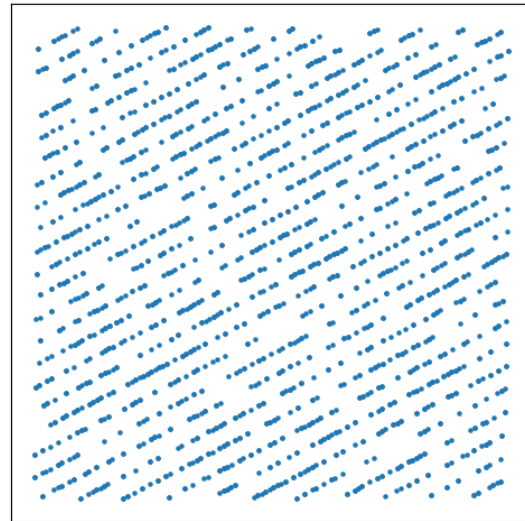
Random Numbers

► Linear Congruential Generator

$$a = 48271, c = 0, m = 2^{32} - 1$$



$$a = 123, c = 456, m = 123456$$



Random Numbers

- ▶ Linear Congruential Generator
 - Download python code from Moodle
 - Scripts -> Linear Congruential Generator
 - Experiment with different values



Random Numbers

- ▶ Better PRNGs
 - XOR Shift
 - Mersenne Twister



Random Numbers

► XOR Shift:

$$X_{n+1} = X_n \oplus (X_n \ll a)$$

$$X_{n+1} = X_{n+1} \oplus (X_{n+1} \gg b)$$

$$X_{n+1} = X_{n+1} \oplus (X_{n+1} \ll c)$$

– Where a, b and c are carefully chosen constants

► Better than LCG, fast



Random Numbers

► Mersenne Twister

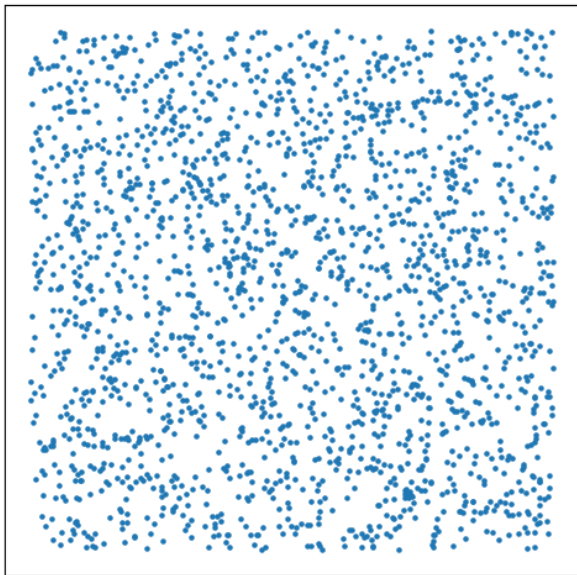
- Matsumoto and Nishimura (1997)
- Uses more state
- Very long period ($2^{19937} - 1$)
- Quite fast, well suited to CPU
- Good randomness, deterministic given seed



Random Numbers

► Mersenne Twister

- Previous states are combined via
 - Bitwise OR
 - Bitshifting
 - Bitmasking
- Good distribution in higher dimensions



Random Numbers

► Mersenne Twister

- In C++ (Sampling.h)
- Base class
 - Allows other sampling method to be implemented

```
class Sampler
{
public:
    virtual float next() = 0;
};
```

Random Numbers

► Mersenne Twister

– In C++ (Sampling.h)

```
class MTRandom : public Sampler {  
public:  
    std::mt19937 generator;  
    std::uniform_real_distribution<float> dist;  
    MTRandom(unsigned int seed = 1) : dist(0.0f, 1.0f) {  
        generator.seed(seed);  
    }  
    float next() {  
        return dist(generator);  
    }  
};
```

Random Numbers

► Mersenne Twister

– In C++ (Sampling.h)

– How to use:

- One per thread `MTRandom *samplers;`
- Initialize

```
SYSTEM_INFO sysInfo;  
GetSystemInfo(&sysInfo);  
numProcs = sysInfo.dwNumberOfProcessors;  
samplers = new MTRandom[numProcs];
```
- Call `samplers[threadID].next();`

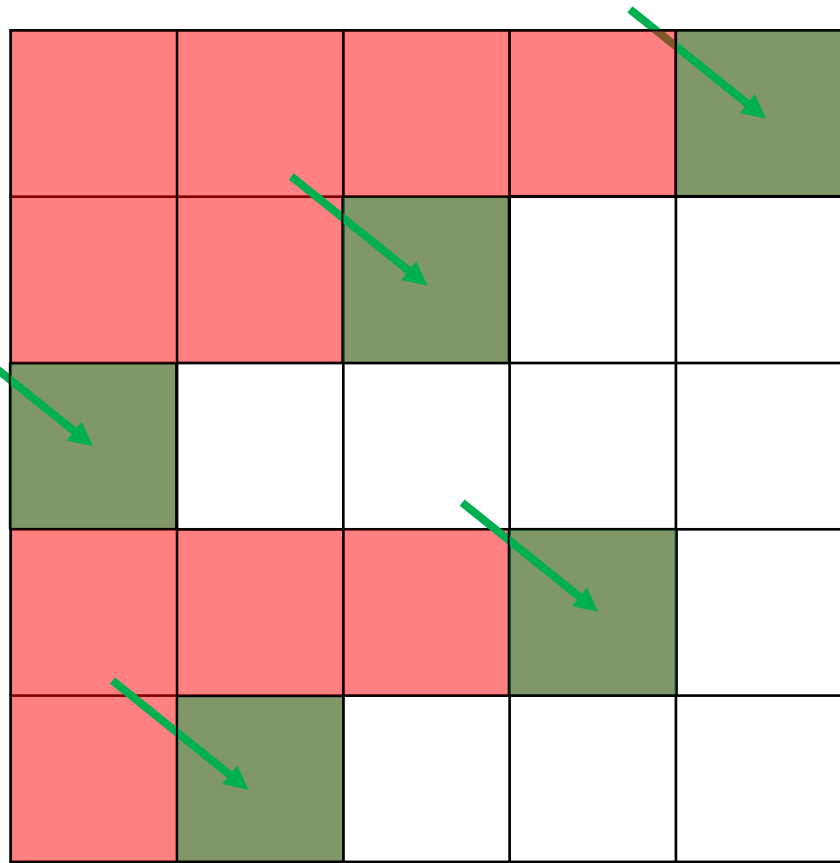
Random Numbers

- ▶ Can we do better?
- ▶ Yes!
- ▶ Generate “random” numbers which are guaranteed to cover the domain
 - Similar to stratified sampling, but without explicit strata



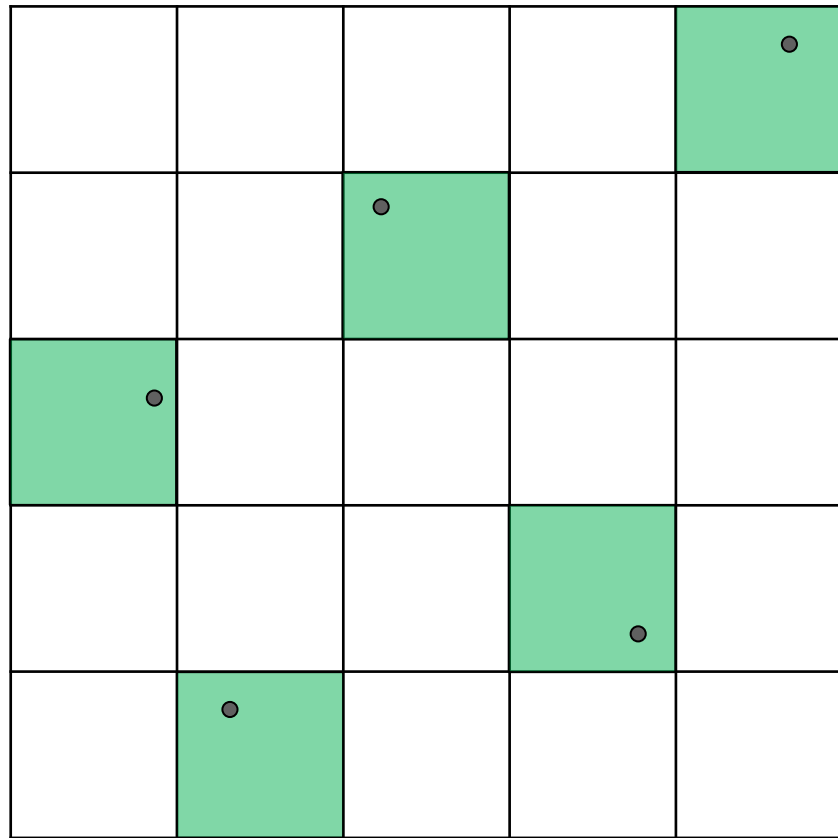
Latin Hypercube

- ▶ Split domain into grid
- ▶ Find cells in grid which match Latin Hypercube criteria
- ▶ Randomly sample in cell (jitter)



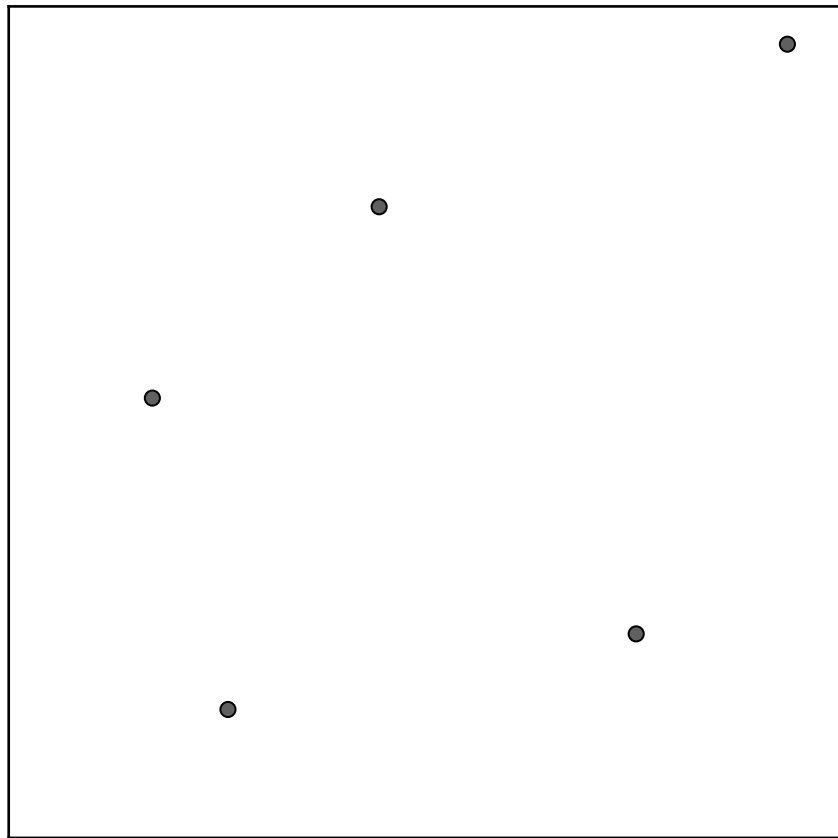
Latin Hypercube

- ▶ Split domain into grid
- ▶ Find cells in grid which match Latin Hypercube criteria
- ▶ Randomly sample in cell (jitter)



Latin Hypercube

- ▶ Split domain into grid
- ▶ Find cells in grid which match Latin Hypercube criteria
- ▶ Randomly sample in cell (jitter)



Quasi Random Numbers

► Discrepancy

- Measures how uniformly a set of points covers the domain $\{x_1, \dots, x_N\} \subset [0, 1]^d$
- Defined as
$$D_N = \sup_B \left| \frac{|\{x_i \in B\}|}{N} - \lambda(B) \right|$$
 - Where B are AABBs and $\lambda(B)$ is the volume of the AABBB

Quasi Random Numbers

► Star Discrepancy

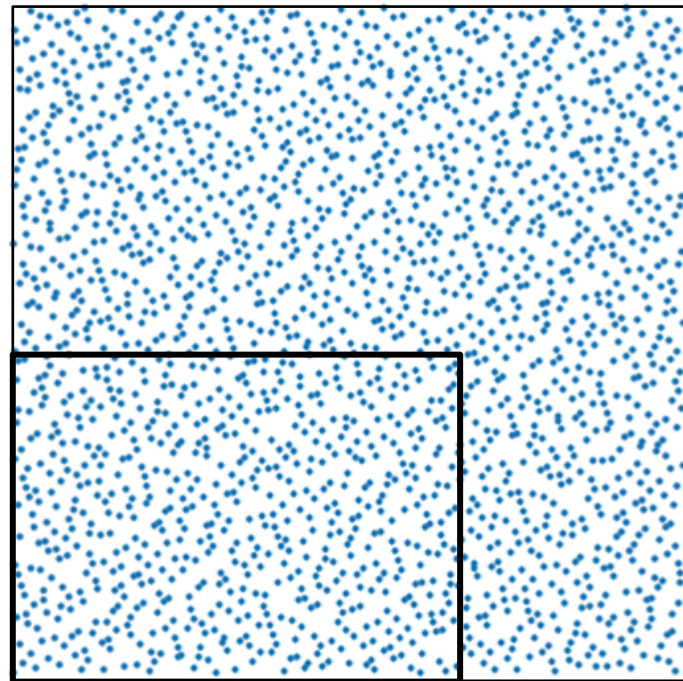
- Measures how uniformly a set of points covers the domain based on anchoring boxes at the origin

$$D_N^* = \sup_{\mathbf{t} \in [0,1]^d} \left| \frac{|\{x_i \in [0, \mathbf{t})\}|}{N} - \prod_{j=1}^d t_j \right|$$

Quasi Random Numbers

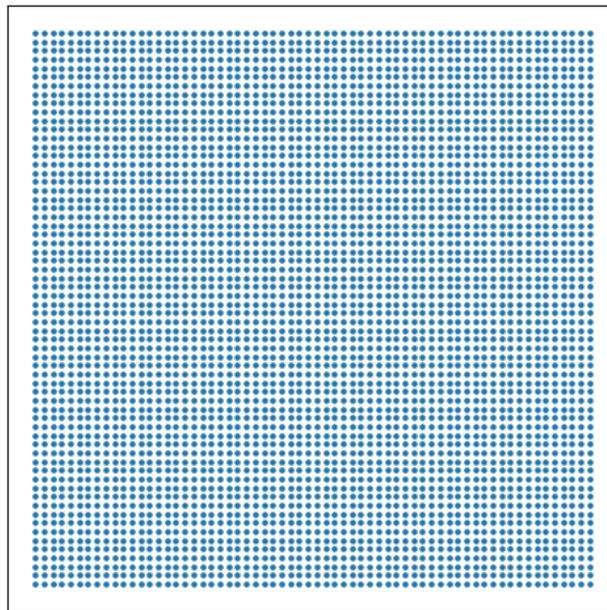
► Star Discrepancy

- Compares the proportion of points in boxes to the box volume
- Measuring the deviation from uniform coverage
- Low values = more uniform



Quasi Random Numbers

- ▶ Grid is one obvious way to achieve this
 - But causes aliasing



Quasi Random Numbers

- ▶ Can create sequences which instead fill in space progressively: Quasi Random Sequences
- ▶ Several
 - Halton
 - Hammersley
 - Sobol etc



Radical Inverse

- ▶ Can express positive integer n in base b over decimal:

$$n = a_0 + a_1 b + a_2 b^2 + \cdots + a_L b^L$$

- Where a are the digits in the base
- e.g.: For base 10 $n = a_0 + a_1 10 + a_2 10^2 + \cdots + a_L 10^L$

Radical Inverse

► Flip over the decimal point:

$$\phi_b(n) = \sum_{j=0}^L \frac{a_j}{b^{j+1}}.$$

- Known as the radical inverse
- e.g.: in base 2

$$\phi_2(n) = \frac{a_0}{2^1} + \frac{a_1}{2^2} + \frac{a_2}{2^3} + \cdots + \frac{a_L}{2^{L+1}}.$$



Radical Inverse

► Known as van Der Corput Sequence

► Discrepancy:

$$D_N^* = O\left(\frac{\log N}{N}\right)$$

– Optimal

n	Binary Representation of n	Radical Inverse in Decimal
0	0	0
1	1	0.5
2	10	0.25
3	11	0.75
4	100	0.125
5	101	0.625
6	110	0.375
7	111	0.875
8	1000	0.0625
9	1001	0.5625

Halton

- ▶ Use radical inverse with a different base for each dimension

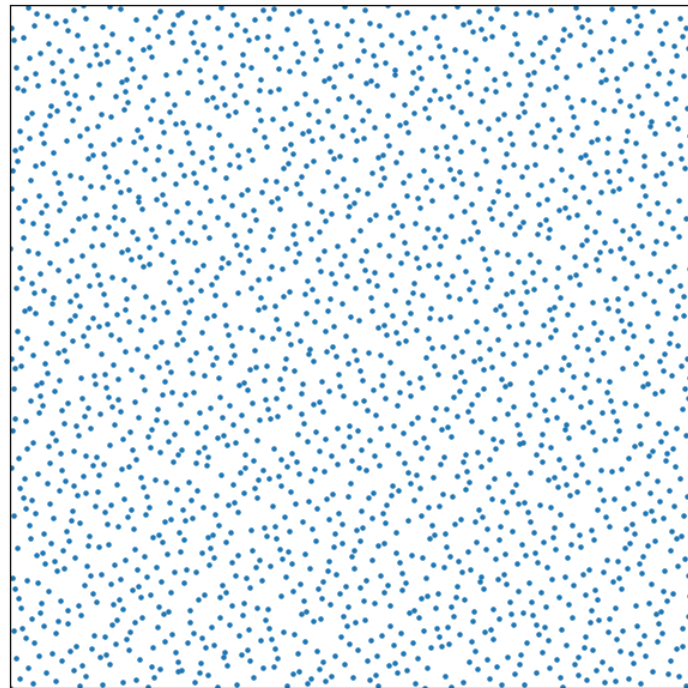
$$\xi_n = (\phi_2(n), \phi_3(n), \phi_5(n) \dots)$$

- ▶ Bases should be prime
- ▶ Close to optimal discrepancy $D_N^* = O\left(\frac{(\log N)^d}{N}\right)$



Halton

- ▶ Good distribution of points
- ▶ Not so fast
 - Needs integer division per digit



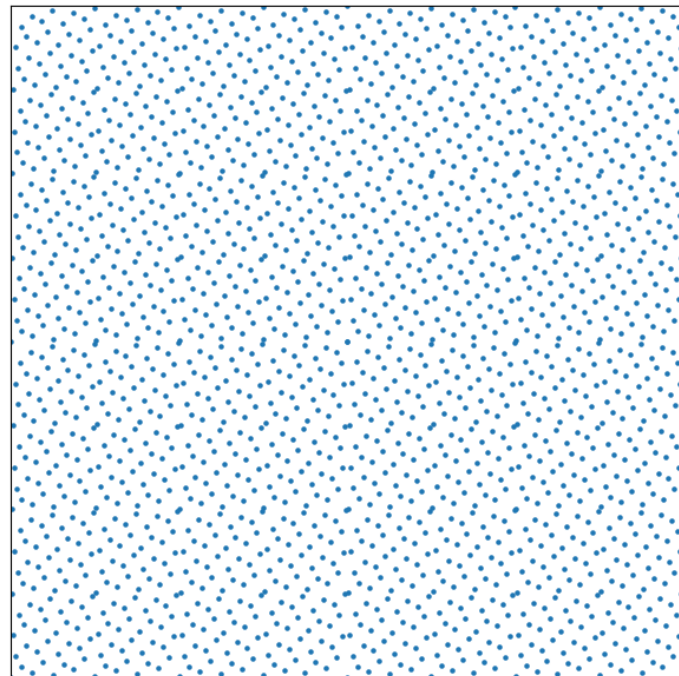
Hammersley

- ▶ Used when number of points is known (N)
- ▶ Linear spread along d=1

- Radical inverse rest

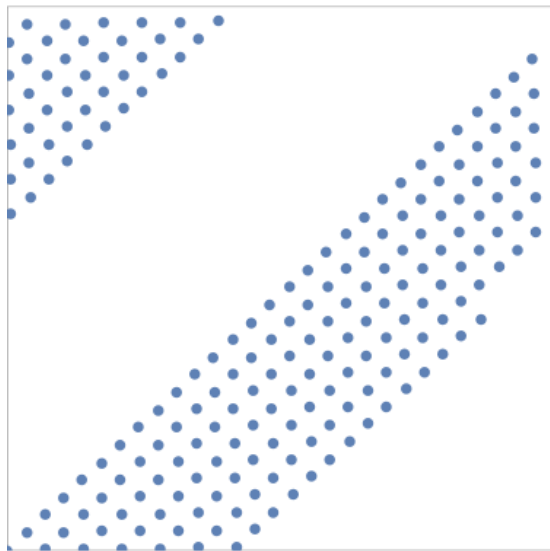
$$\xi_n = \left(\frac{n}{N}, \phi_2(n), \phi_3(n), \dots \right)$$

- ▶ Slightly better discrepancy

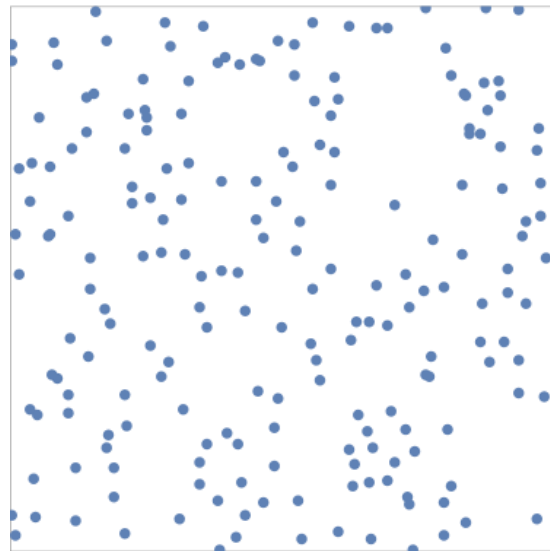


Scrambling

- ▶ Correlations can still appear between dimensions
- ▶ Permute digits
 - Random
 - Owen



(a)



(b)

Sobol

► Only use base 2 (fast)

► So $n = \sum_{k=0}^L a_k 2^k, \quad a_k \in \{0, 1\}$

► Define digit vector $a(n) = \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_L \end{bmatrix}$

► Multiply by Generator matrix G : $y = G a(n) \pmod{2}$

► Output $\xi_n = \sum_{i=1}^L y_i 2^{-i}$



Sobol



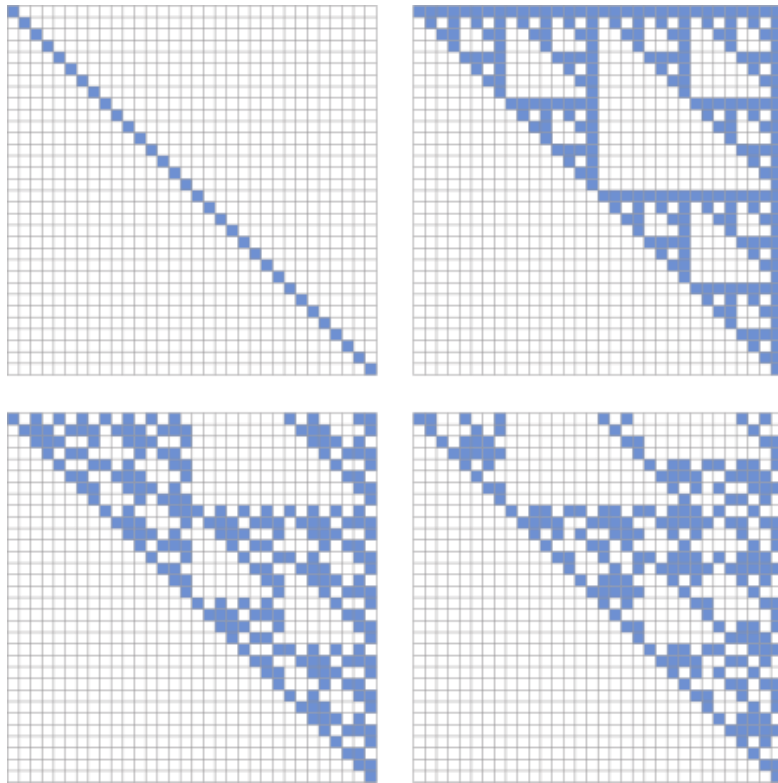
Generator matrix

$$\begin{bmatrix} g_{1,1} & g_{1,2} & \cdots & g_{1,L} \\ 0 & g_{2,2} & \cdots & g_{2,L} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & g_{L,L} \end{bmatrix}$$

- where $G \in \mathbb{F}_2^{L \times L}$, $g_{i,k} \in \{0, 1\}$
- L is 32 or 64 (float or double)

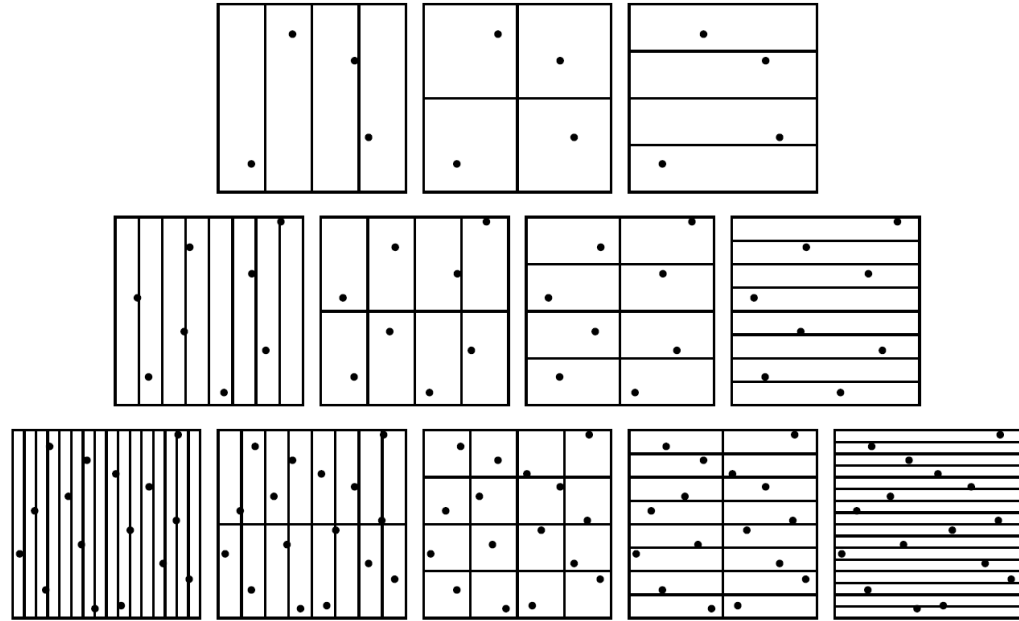
Sobol

- ▶ Different generator matrix for each dimension
- ▶ Active research field



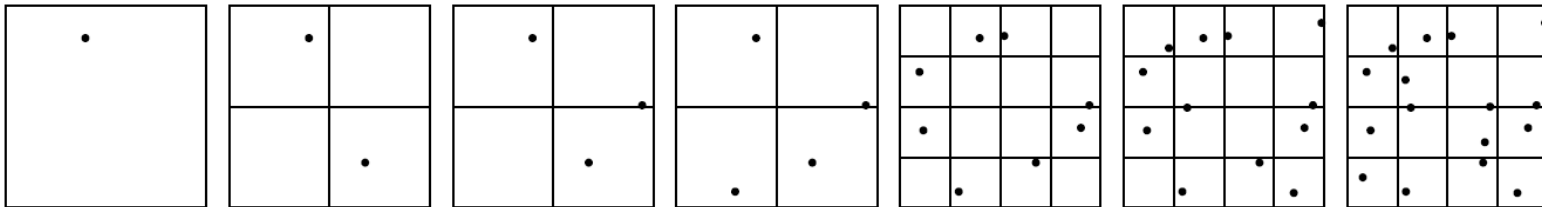
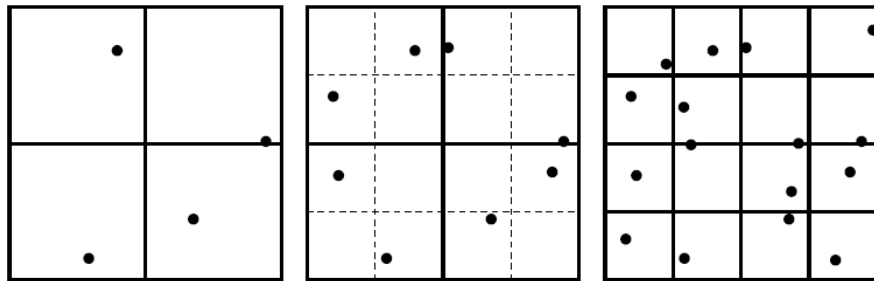
Multi Jittered Sampling

- ▶ One sample per cell for any discretisation of domain



Multi Jittered Sampling

► Can be progressive



Poisson Disk Sampling

▶ Blue noise

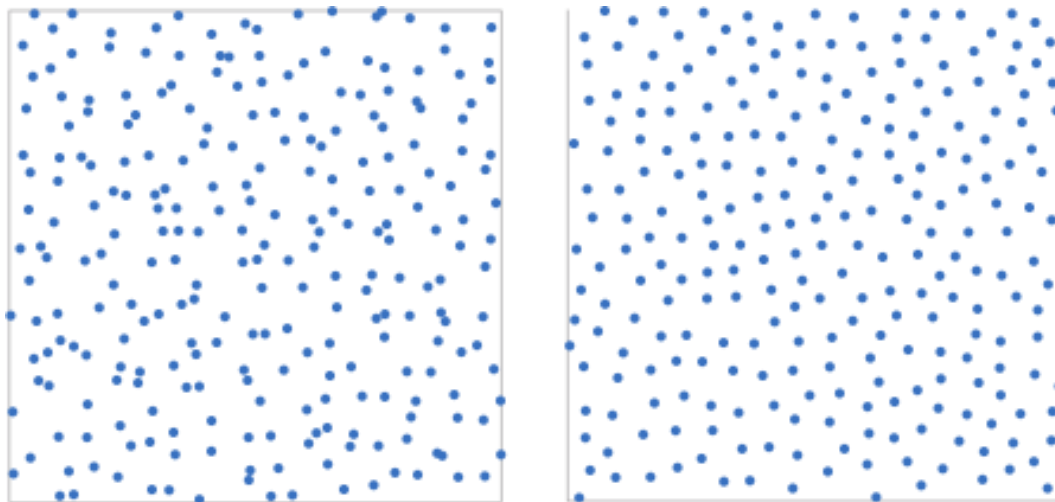
▶ Algorithm:

- 1. Start with set containing one random point
- 2. Generate new point
- 3. Accept if not close to any elements in set
- 4. Goto 2. until domain filled



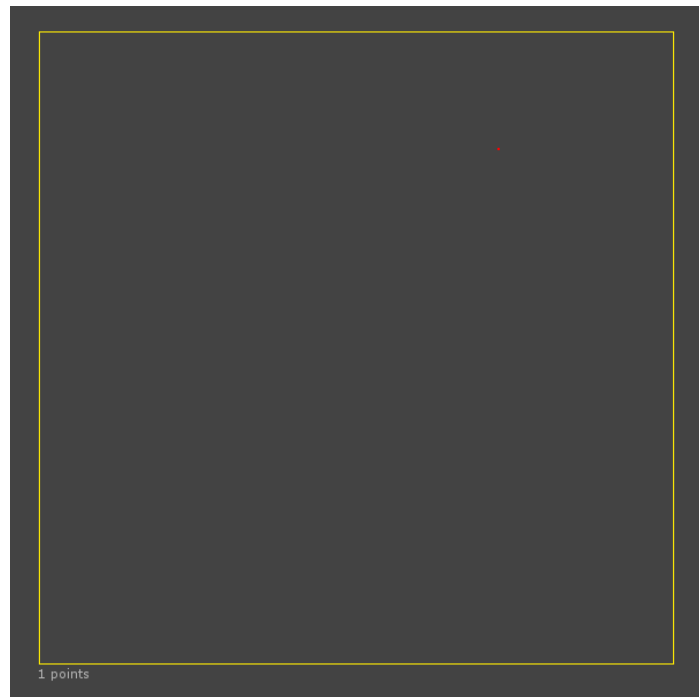
Poisson Disk Sampling

- ▶ Good distribution of points and well spaced



Poisson Disk Sampling

- ▶ Can be optimised to generate points near previous points
 - Bridson 2007
- ▶ Much faster
 - But too slow for most Monte Carlo



Quasi Random Sequences

► [Web app](#)



Use in Monte Carlo

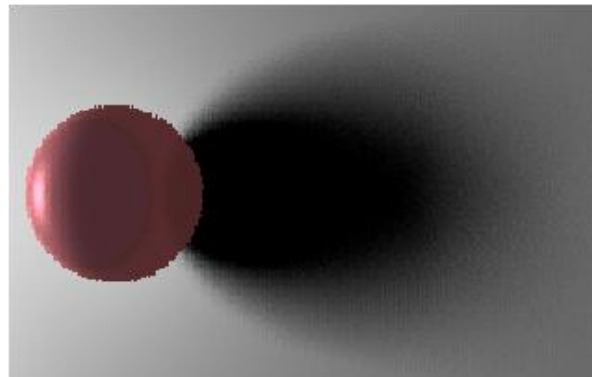
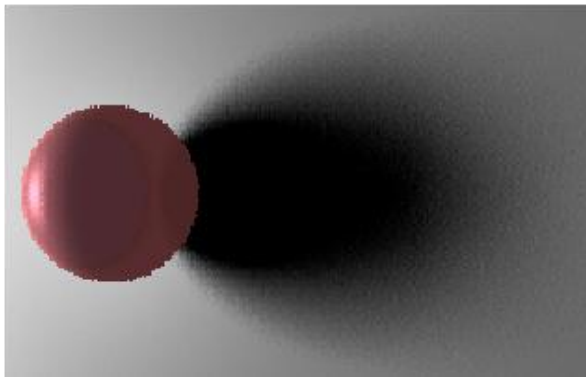
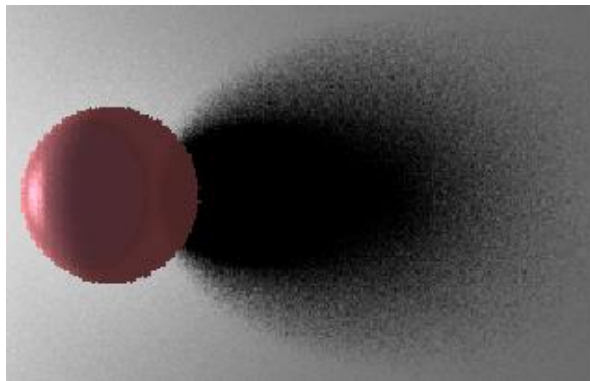
► Quasi Monte Carlo

- Deterministic
- Better convergence, especially in lower dimensions
 - Often near $\mathcal{O}(\frac{1}{N^{1-\epsilon}})$, rather than $\mathcal{O}(\frac{1}{\sqrt{N}})$
- More sensitive to discontinuities (rendering!)



Quasi Monte Carlo

- ▶ Can substantially reduce variance
 - Used in games and films



Summary

- ▶ Introduced Monte Carlo Integration
- ▶ Discussed Variance Reduction methods
- ▶ Discussed random number generation
- ▶ Discussed quasi random numbers and QMC



Next Steps

- ▶ Work on sampling tabulated distribution
 - Implement tabulated marginal and conditional distributions and sampling
 - Part of assignment for sampling environment maps





Questions?

Computer Graphics

Monte Carlo Integration



WARWICK

Dr. Tom Bashford-Rogers